DumosRx

Complete Feature Documentation

All 11 app sections, walked live against real seeded data and documented section by section.

Generated 2026-09-03

Contents

1. 1. Dashboard
2. 2. Inventory
3. 3. Point of Sale
4. 4. Prescriptions
5. 5. Customers
6. 6. Procurement
7. 7. Expenses
8. 8. Reports
9. 9. Activity Log
10. 10. Settings
11. 11. Authentication

Dashboard

Route: `app/(dashboard)/dashboard/page.tsx` → `components/dashboard/dashboard-overview.tsx`. Business logic lives in `lib/hooks/use-dashboard-overview.ts` (stats, sales comparison, recent-activity feed) and `lib/hooks/use-stock-batch-stats.ts` (inventory-derived counts).

Walked live against the "Pikarestiv Stores 2" store on 2026-09-02.

Today's Sales stat card

Shows the store's net revenue for the current calendar day (`todayRevenue = salesToday.total - refundsToday.total`, computed in `useDashboardOverview`), formatted with `formatMetricCurrency` (rounds to whole Naira — no kobo). Backed by `getDashboardOverviewData()` (`lib/db/queries/reports.ts`) via `useQuery(queryKeys.dashboard.overview(viewerId))`. Below the figure, `renderSalesComparison()` shows "X% vs yesterday" (up/down arrow) or "No sales yesterday" when `salesYesterday` is 0 — this store showed ₦0 today and "No sales yesterday", correctly rendering the `state: "none"` branch. Non-admin roles are scoped to their own sales only (`viewerId` is set unless `checkCanViewAllActivity(user.role)`).

Total Products stat card

Shows `stock_batchStats.activeProducts` and, in the caption, "Across N categories" (`activeCategories`). Backed by `useStockBatchStats()` → `getStockBatchStats(expiryDays)` (`lib/db/queries/inventory.ts`), the single source of truth the hook's own doc-comment says all inventory stat cards across the app should use. Live: 1,513 products across 10 categories.

Inventory Value stat card

Shows `stock_batchStats.totalStockBatchValue`, formatted with `formatMetricCurrency` — same query as Total Products (`getStockBatchStats`). Caption is a static "Calculated stock value" (not a comparison). Live: ₦1,929,051.

Orders Today stat card

Shows `salesToday[0]?.count || 0` — completed-transaction count for today, from the same `getDashboardOverviewData()` payload as the Today's Sales card. Live: 0 (matches ₦0 in Today's Sales).

Action Center

`components/dashboard/dashboard-action-center.tsx`. Renders only for admin roles (`isAdmin`), and only when at least one alert applies — it's absent entirely for a fully-configured store with no admin, and it auto-scrolls horizontally every 5s (paused on hover/touch) when there's more than one card. Alerts observed live, each a clickable card that `router.push()`s to `actionRoute`:

- **Trial (N Days Left)** — from `checkLicenseStatus()` (`lib/licensing/licensing-manager.ts`), links to `/settings/billing`.
- **N Items Low Stock** — from `lowStockCount` (passed down from `stock_batchStats`), links to `/inventory/catalog?status=low_stock` (see Resolved bug below — the catalog tab genuinely honors the `status` query param and pre-filters to the "Low Stock" chip).
- **N Items Oversold** — from `oversoldCount` (sourced from `getOversoldAlerts()` directly in `useDashboardOverview`, not from `stock_batchStats`), links to `/inventory/catalog?status=out_of_stock`. `priority: "critical"` (destructive/red styling, distinct `AlertOctagon` icon) — separate from and additive to the Low Stock card; fixes bug #9 (see Resolved below).
- **N Batches Missing Expiry** — from `missingExpiryCount`, links to `/inventory/overview` (confirmed working live).
- **Profile N% Complete** — from how many of `name/address/phone/email/logo_url` (+`pcn_license` for pharmacies) are filled on `storeProfile`, links to `/settings/store`.
- **Get the App** — shown to store owners not already on Tauri/PWA, links to `/settings/system`. Other possible alerts not observed live (conditions weren't met on this store): No Cloud Account, Subscription Expired/Expiring, No Staff Accounts, N Changes Unsynced (`getSyncQueueCount()`, `queryKeys.sync.queueCount()`, polled every 5s).

Recent Activity

`components/dashboard/dashboard-recent-activity.tsx`. Shows the 5 most recent entries from `dashboardData.recentActivities` (part of the same `getDashboardOverviewData()` query as the sales stat cards). Each row is clickable and opens a details dialog scoped to its `activity_type`: sale → `TransactionDetailsDialog`, expense → `ExpenseDetailsDialog`, purchase_order → `ProcurementDetailsDialog`, prescription → `DashboardPrescriptionDetailsDialog`, stock_movement → `StockMovementDetailsDialog`. **Caveat observed live:** all 5 rows on this store were "Product added" (`activity_type: "product"`) entries, and clicking any of them does nothing — `dashboard-overview.tsx` never renders a dialog for `selectedActivity?.type === "product"`, so the click sets state but no UI reacts. Not fixed as part of this task (a `product` details dialog would be new UI, not a broken-link/wrong-number regression), but worth a follow-up ticket. "View All" (top-right) navigates to `/reports` (confirmed live) — not to an activity-log filtered view.

Quick Actions

`components/dashboard/dashboard-quick-actions.tsx`. Static, role-filtered link grid (`adminOnly` actions hidden unless `isAdmin || canManageStockBatch`): New Sale → `/pos`, Add Stock → `/procurement/new` (admin-only), Close Register → `/reports?tab=daily_close`, Scan Barcode → `/pos?action=scan`, Customers → `/customers`, View Reports → `/reports` (admin-only), Reorder Stock → `/procurement` (admin-only). "New Sale" confirmed live to navigate to `/pos`; the rest are plain `next/link` hrefs to existing routes, not independently re-verified.

Notifications bell

Top-right bell icon opens a dropdown titled "Notifications" listing recent activity-log entries (e.g. "LOGIN — Action: LOGIN on users", "INSERT — Action: INSERT on products") with relative timestamps. This is a raw audit-log feed, not the same data as the Recent Activity card (which is curated by business activity type, not raw DB action).

Loading state

`DashboardOverview` renders a skeleton grid (4 stat-card skeletons + 2 large panel skeletons) while `isLoading` — defined as `stock_batchStats.loading && !salesToday.length`. Not independently reproduced live (data loaded before a screenshot could be taken), but confirmed by reading the component.

Resolved

"N Items Low Stock" Action Center card crashed the whole app

- **Found:** clicking the "473 Items Low Stock" card navigated to `/inventory/products?status=low_stock`. `/inventory/[tab]/page.tsx` only pre-renders `overview`, `catalog`, `batches`, `ledger`, `audits` via `generateStaticParams()` (required because the app builds with `output: export`); `products` isn't one of them, so Next.js threw a full-page "Runtime Error: Page ... is missing param ... in generateStaticParams()" instead of navigating. The same dead route is also referenced from `components/stock-batch/needs-attention.tsx`'s "View all" link (out of scope for this Dashboard task — not fixed, flagged here for a future Inventory-section pass).
- **Fix:** `components/dashboard/dashboard-action-center.tsx` now points the low-stock alert at `/inventory/catalog?status=low_stock` — the "catalog" tab is the actual product list, and it turns out it already reads and applies the `status=low_stock` query param (shows the "Low Stock" filter chip pre-applied), so the fix restores the intended filtered view, not just a non-crashing page.
- **Verified by:** `client/__tests__/dashboard-action-center-routes.test.ts` — parses `dashboard-action-center.tsx` and `inventory/[tab]/page.tsx`, asserts every `/inventory/<tab>` actionRoute names a tab that actually exists. Confirmed to fail (`references`

`unknown inventory tab "products"`) against the original code, pass after the fix. Also re-clicked the card live post-fix: lands on the Catalog tab with the Low Stock chip active, no error overlay.

Dashboard's Action Center had zero signal for an oversold/floored product (bug #9)

- **Found:** review of bug #4's fix — a product floored to `quantity=0` (bug #4's "floor + alert" fix) failed `getStockBatchStats()` 's `low_stock_count` SQL's `total_qty > 0` guard, so it produced no signal anywhere on the Dashboard.
- **Fix:** added a new, separate "Oversold" card (see the "N Items Oversold" entry above) rather than changing `low_stock_count` 's existing SQL/guard, to avoid any silent behavior shift for other consumers of that boundary.
- **Verified by:** `client/__tests__/dashboard-action-center-routes.test.ts` — new case asserting `oversoldCount` is threaded through `dashboard-overview.tsx` , the card is conditional on `oversoldCount > 0` , and its route is `/inventory/catalog?status=out_of_stock` . See `### 17.` in `_findings-log.md` for full detail.

Inventory

Routes: `app/(dashboard)/inventory/page.tsx` (renders `overview` directly, without redirecting the URL) and `app/(dashboard)/inventory/[tab]/page.tsx` (the tab-switched shell) → `components/stock-batch/stock-batch-management.tsx`. `generateStaticParams()` on the `[tab]` route pre-renders exactly five tab values — `overview`, `catalog`, `batches`, `ledger`, `audits` — because the app builds with `output: export`; any other value 404s or (if linked to from inside the app) throws a full-page Next.js runtime error instead of navigating. Of those five, only three are exposed as actual clickable tabs in the UI (`StockBatchTabNav`): **Overview**, **Catalog**, and **Movements** (the visible label for the `ledger` route — gated behind `canManageStockBatch`). `batches` has no dedicated screen of its own; `audits` isn't a tab either — visiting `/inventory/audits` immediately flips `isAuditing` to `true` and redirects to `/inventory/overview` (`lib/hooks/use-stock-batch-management.ts`), so the audit flow always overlays on top of Overview.

Walked live against the "Pikarestiv Stores 2" store (1,513 products, 10 categories, ₦1,929,051 stock value, 473 low-stock) on 2026-09-02.

Overview tab

`components/stock-batch/stock-overview.tsx`, backed by `useStockBatchStats()` and `getStockOverviewData()` (`lib/db/queries/inventory.ts`) — the same stats source Task 1's Dashboard walkthrough documented. Four stat cards (Total stock value, Total products, Low stock, Expiring soon) sit above two panels:

- **Needs attention** (`needs-attention.tsx`) — up to 10 of the worst items: expiring batches (`getExpiringBatches(expiryDays)`, "View batch" → `/inventory/batches`) and low/critical/out-of-stock products ("Reorder" → `/procurement`). Live: showed "(10 of 50)" with an out-of-stock item first. **"View all" bug, fixed this task** — see Resolved section below.
- **Fast movers** (`fast-movers.tsx`) — last-7-days top sellers; empty live (this store has zero recorded sales/movements yet).

"Start Audit" (admin-only) is a header-level button here, not a tab — see Audits below.

Catalog tab

`components/products/product-database.tsx` (via `ProductDatabase` → `ProductDatabaseFilters` + `catalog-list.tsx`). Route title: "Product Catalog".

- **Search** — `SearchInput`, live-filters by name/SKU. Confirmed live: typing "paracetamol" narrowed 1,513 rows to the ~10 paracetamol products.
- **Category filter pill** — populated from `getCategoriesList()` (`lib/db/queries/products.ts`), which scopes to the active store (`WHERE store_id = ?`). See **Open finding** below: on Store 2 this returned Groceries/Beverages/Personal Care/Household/Snacks/Dairy — the generic non-

pharmacy fallback list, not the store's real categories (Drugs, Cosmetics, etc. — visible on every product row). Selecting one of the fallback categories (e.g. "Groceries") correctly shows "No products found", so the filter mechanism itself works; only the option list is wrong.

- **Inventory filter pill** — All / Active / Inactive / Expiring Soon / Expired / Low Stock / Out Of Stock. Confirmed live: "Low Stock" narrows to the 473 below-reorder products and sets the `Inventory: Low Stock` chip — this is the exact filter state the Dashboard and Overview "low stock" links land on via `?status=low_stock`.
- **Column sort** — every header (Product, Category, Avg Cost, S. Price, Stock, Reorder) is clickable with an ascending/descending toggle. Confirmed live: sorting by Stock ascending correctly re-ordered to all-zero-stock rows first.
- **Manage** (`manage-categories-dialog.tsx`, admin-only) — a small dialog to add/rename/delete categories, plus an "Add starter categories" button that seeds `DEFAULT_CATEGORIES` (`lib/db/queries/categories.ts`: Drugs, Beverages, Toiletries, Cosmetics, Perfumes, Wines & Spirits, Provisions, Biscuits, Tea, Groceries, Baby Care). **Its list did not match the Category filter pill's list** live (see Open finding below) — it showed Analgesics/Antacids/Antibiotics/Antidiabetics/Antihistamines/ Antihypertensives/Antimalarials, because it reads via a *different*, **store-unscoped** query (`getCategoryList()` in `lib/db/queries/categories.ts`, no `store_id` filter at all) than the filter pill's store-scoped `getCategoriesList()` (`products.ts`).
- **Export** — 4 options: CSV/XLSX × all-columns/choose-columns (`export-columns-dialog.tsx`, `buildExportBlob()` in `lib/utils/product-import-export.ts`). Confirmed live: "Export as CSV (all columns)" produced a toast "Exported 1513 product(s)" and downloaded a file.
- **Import** — see its own section below; this is the flow that shipped two real production bugs (see `docs/features/_findings-log.md` entries #1–2) with zero prior e2e coverage.

Import flow (multi-sheet workbook support)

`components/stock-batch/import-mapping-dialog.tsx` (`ImportMappingDialog`), triggered from `import-export-toolbar.tsx`'s "Import" button. State machine: `pick-file` → `pick-sheet` (only if the workbook has >1 sheet) → `map-columns` → `importing` → `result`.

1. **Pick file** — accepts `.csv,.xls,.xlsx`. `readWorkbookFile()` (`lib/utils/product-import-export.ts`) parses it via the `xlsx` package.
2. **Sheet picker** — shown only when `workbook.SheetNames.length > 1`: "This file has N sheets. Select the one with your product list." Single `Combobox` populated from every sheet name in the file, in file order. Confirmed live and in `e2e/product-import.spec.ts` with a synthetic 3-sheet fixture (Sheet1: 3 rows, Sheet2: 2 rows, Sheet3: 0 rows) — all three sheet names appear as selectable options, and picking one loads that sheet's own row count into the next step.
3. **Auto-mapping** (`detectColumnMapping()`) — matches headers against a `HEADER_ALIASES` table seeded from real QuickBooks POS / Moniebook / DumosRx export headers (e.g. "Item Number"→barcode, "Item Name"→name, "Average Unit Cost"→cost_price, "Regular Price"→selling_price, "Department Name"→category, "Qty 1"→quantity). Header text is

case/whitespace-normalized and strips bracket/percent suffixes (`normalizeHeader()`) before matching, and each internal field can only be auto-claimed by the *first* matching column (later duplicates fall back to "Ignore this column") since DumosRx tracks one quantity per product, not one per branch. Shows "We matched N of M columns automatically. Review or correct any below," with a per-column `Combobox` to override. Confirmed live and in the e2e spec: the fixture's 6 QuickBooks-shaped headers auto-matched 6 of 6, with the correct row count ("3 row(s) will be imported" / "Import 3 Row(s)" button) tracking whichever sheet was picked.

4. **Dedupe warning** — `findInFileDuplicates()` (`lib/db/queries/product-import.ts`), triggered on clicking "Import N Row(s)": if the file has more than one row with the same product name *and* category, a `window.confirm()` warns "N product name(s) appear more than once in this file with the same category. Importing anyway will merge them into one product (last row wins). Continue?" before proceeding. Not exercised live this session (the fixture has no in-file duplicates) — logic-level coverage only, via Vitest.
5. **Importing / Result** — `importProductRows()` writes rows in a single batched `transaction()` (the fix for finding #1 in the log) and reports `{created, updated, skipped}`, with skip reasons listed per row.

Not driven to completion live this session by design: Step 1 explicitly scopes this to "open the dialog, verify the sheet picker, then close without importing" to avoid re-polluting Store 2's real 1,513-row catalog with the synthetic fixture data. The write path itself (`importProductRows` , batched invalidation) is unchanged from the fix verified in finding #1's 323-passing-test / live 1,513-row re-import verification.

Movements tab (`ledger` route)

`components/stock-batch/stock-movements.tsx` . Route title: "Stock Movements" (gated behind `canManageStockBatch` ; direct navigation to `/inventory/ledger` without that permission bounces to `/inventory/overview` per `use-stock-batch-management.ts`). An immutable log ("Immutable log, entries can't be edited") of stock movements with:

- **Search** by product, reference, or user.
- **Type filter pills** — All types / Sales / Restock / Returns / Damage / Adjustments.
- **Date range picker** — presets (Today, Yesterday, Last 7 days, Last 30 days, This month, Last month, Year to date, Last year) plus a manual two-month calendar range. Defaults to "last 30 days."
- **Sortable columns** — Time, Product, Type, Qty, Reference/Reason, User.

Confirmed live: default (last-30-days) view showed "No movements found," and widening to "Last year" (1 Jan–31 Dec 2025) was still empty — this store genuinely has zero recorded stock movements yet (the 1,513-product bulk import writes products/batches directly, not through the movements ledger, and there have been no sales/restocks/audits submitted on this store to date).

Audits ("Start Audit" flow)

Not a tab (see routing note above) — `components/stock-batch/stock-audits.tsx`, a full-screen overlay opened by the Overview tab's admin-only "Start Audit" button. Three-step flow (`AuditStep`: `ledger` → `review` → `done`):

1. **Physical inventory (ledger step)** — on open, immediately triggers a full `sync(true)` before showing any rows ("Syncing latest stock levels…"), so counts always reflect the latest known state, not a stale local snapshot. Confirmed live: after syncing, showed "1513 of 1513 items shown," one editable row per product (Counted Qty, Counted Cost, Counted Selling — each pre-filled with the system's current value via `EditableNumberCell`), with live Diff Qty/Diff Cost/Diff Selling columns and a category filter + search. Every field starts equal to the system value, so nothing counts as "adjusted" until a value is actually touched.
2. **Review & submit** — `audit-review-step.tsx`; shows a "Total Counted" / "Adjusted" summary and a card per adjusted item with its qty change (confirmed live: editing one product's Counted Qty from 0→5 surfaced exactly one card, "MACA GUMMIES — Qty: 0 → 5", with Adjusted: 1).
3. **Submit** — `useSubmitStockAuditMutation()` writes one stock-movement row per adjusted item (`reason` supported per-row) and shows a "Cycle Count / Finished" success screen with counted/adjusted totals.

Verified the full ledger→review round trip live on Store 2's real data, then **backed out without submitting** (reset the edited quantity to 0 and navigated away) specifically to avoid writing a synthetic adjustment into the real audit/movement history, per this task's no-destructive-writes constraint. The submit path itself is otherwise unverified live this session — only its Vitest-level and manual pre-submit-screen behavior are confirmed.

Resolved

"Needs attention" panel's "View all" link crashed the app

- **Found:** clicking "View all" on the Inventory Overview tab's Needs Attention panel navigated to `/inventory/products` — the identical dead route Task 1 found and fixed in the Dashboard's Action Center (`/inventory/products?status=low_stock`), left as a known residual issue for whichever task walked Inventory (see finding #3 in `docs/features/_findings-log.md`). Reproduced live: full-page Next.js "Runtime Error: Page ... is missing param ... in generateStaticParams()".
- **Fix:** `client/components/stock-batch/needs-attention.tsx`'s "View all" `onClick` now calls `router.push("/inventory/catalog?status=low_stock")` — the same real, working destination Task 1's fix used, which the Catalog tab already reads and applies via its Inventory filter pill.
- **Verified by:** a second assertion added to the existing `client/__tests__/dashboard-action-center-routes.test.ts` (this file doesn't use the `actionRoute:` prop pattern Task 1's regex targets — it calls `router.push()` directly inline — so the new assertion uses its own regex against the same `allowedTabs` source of truth). Confirmed to fail (`references unknown inventory tab "products"`) against the pre-fix source via `git stash`, pass after. Re-clicked "View all" live post-fix: lands on Catalog with the Low Stock chip pre-applied, no error overlay.

Category filter pill and Manage Categories dialog disagreed, and neither showed this store's real categories

- **Found while walking Catalog:** the Category filter pill's option list (Groceries/Beverages/Personal Care/Household/Snacks/Dairy) doesn't match either the categories actually used by Store 2's products (Drugs, Cosmetics, and others — visible on every product row) or the list shown in "Manage Categories" (Analgesics/Antacids/Antibiotics/Antidiabetics/Antihistamines/Antihypertensives/Antimalarials — likely another store's categories, or an orphaned starter-category seed).
- **Root cause:** two different, non-equivalent queries both read the same `categories` table. The filter pill uses `getCategoriesList()` (`lib/db/queries/products.ts`), which correctly scopes `WHERE store_id = ?` — but returned zero rows for this store, triggering product-database.tsx's hardcoded `defaultCategories` fallback. "Manage Categories" used a *different* function, `getCategoryList()` (`lib/db/queries/categories.ts`), which had **no `store_id` filter at all** — a genuine multi-tenancy leak, since it could show and let a store owner edit/delete categories belonging to a different store.
- **Fixed (`be38864e`):** `getCategoryList()` now filters on `store_id` (rows from before the fix with a NULL `store_id` stay visible to every store, so pre-existing data isn't hidden), and `createCategory()` now explicitly sets `store_id` . See bug #1 in `_known-bugs.md` and `### 12.` in `_findings-log.md` for the full investigation, including a flagged follow-up: `update()` / `softDelete()` in `base-helpers.ts` do no `store_id` ownership check, a latent cross-tenant risk shared by every domain table (not category-specific), left unfixed here.
- **Separately, still open:** *why* the store-scoped `getCategoriesList()` returned zero rows for Store 2's real categories (triggering the fallback list) wasn't investigated as part of this fix — it needs a look at the `categories` table's actual `store_id` population/backfill history for that account, independent of the query-scoping bug above.

Open

Stock Movements tab is untestable against a store with zero movement history

- **Found while walking Movements:** Store 2 has recorded zero stock movements (bulk product import doesn't route through the movements ledger, and no sales/restocks/audits have been submitted on this store). This means the Movements tab's search, type filters, and sort could only be confirmed to render an empty state correctly, not that they actually filter/sort real rows.
- **Coverage:** the new `e2e/product-import.spec.ts` doesn't cover this tab either (out of scope — see Step 4). No existing e2e spec drives the Movements tab at all.
- **Not fixed:** would require either seeding movement rows into a test store or running a real sale/restock against Store 2's live data, which this task's no-destructive-writes constraint rules out for the real store. Recommend adding movement-row fixtures to the Playwright test-db seed (`e2e/global.setup.ts`) so a future spec can assert real filter/sort behavior.

`client/e2e/products.spec.ts` and its shared `login()` fixture had drifted out of sync with the current UI

- **Found while running Step 3/6 verification:** `e2e/fixtures.ts`'s shared `login()` helper used `page.getByPlaceholder('••••')` for the PIN field, but `components/auth/traditional-login-form.tsx` switched that field to an `InputOTP` component with no such placeholder — every spec using `login()` (all of them) was silently timing out on login. **Fixed** as part of this task (`e2e/fixtures.ts` now uses `input[data-input-otp="true"]`, the same selector `global.setup.ts` already used for the analogous field on `/setup`), since it blocked this task's own required `npx playwright test` verification step.
- **Found, not fixed:** `products.spec.ts` itself has two further stale assertions surfaced once login was fixed: (1) it expects `/inventory` to redirect to `/inventory/overview`, but `app/(dashboard)/inventory/page.tsx` renders the Overview tab directly without changing the URL — this may be a legitimately stale test assertion rather than a product bug; (2) it expects the `/inventory/ledger` header to read "Stock Ledger", but `lib/constants/dashboard-page-routes.ts` titles that route "Stock Movements". Both are pre-existing, unrelated to any file this task touched, and out of scope to fix here.
- **Found, not fixed (larger issue):** `e2e/global.setup.ts` (which builds the shared `.auth/test-db.bin` fixture every other spec's `login()` restores) is also broken against the current `/setup` wizard — it clicks a button named "Create New Store" that no longer exists (now "Set Up New Business"), and even past that, `register-step.tsx` now requires Email/Phone/Password/Confirm Password fields the setup script never fills. This is a substantial, pre-existing drift affecting every e2e spec in the suite, not specific to Inventory; fixing it needs a product decision on what test credentials to seed and was left out of scope here. This task's own verification instead reused the already-committed `e2e/.auth/test-db.bin` fixture via `--no-deps`, bypassing the broken setup step (see task report for exact commands run).

Point of Sale (POS)

Route: `app/(dashboard)/pos/page.tsx` → `components/pos/pos-system.tsx` (`POSSystem`), orchestrated by `lib/hooks/use-pos-system.ts` and its sub-hooks (`use-pos-cart.ts` , `use-pos-payment.ts` , `use-pos-held-transactions.ts` , `use-pos-data.ts` , etc.).

Walked live against the "Pikarestiv Stores 2" store on 2026-09-02, including real checkouts (cash + transfer) and a real hold/recall cycle — this store is the plan's designated real-write sandbox for POS, unlike the read-only walkthroughs of prior tasks.

Layout

Two-pane desktop layout (`POSLayoutHeader` + product grid on the left, `POSCartPanels` cart on the right; collapses to a bottom drawer on mobile via `POSMobileCartDrawer`). Two main tabs: **Products** and **Recent Sales** (`POSMainTabNav` / `POSTransactionHistory`).

Product search & selection

- **Search box** (`POSLayoutHeader` , header + a duplicate mobile-row input) — live-filters by name/SKU as you type. Confirmed live: searching "syringe" correctly narrowed the grid to the three syringe SKUs (10ML/2ML/5ML) and showed the 5ML one as "Out of stock" (correctly not addable to cart).
- **Category filter pills** (`POSCategoryFilter`) — All + one pill per category present in this store's catalog (Beverages, BUSICUIT, Cosmetics, CREAMS, Drugs, machine, PERFUME, PROVISION, System, Toiletries, …).
- **"Goes well with cart" smart suggestions** — appears once the cart has at least one item; suggests related/frequently-co-purchased products. Confirmed live: adding a syringe surfaced other syringe sizes and creams as suggestions.
- **"Recently sold" row** — appears on the Products tab once at least one sale has completed this session, showing the just-sold products with an updated stock badge.
- **Scan** button opens `CameraScannerDialog` for barcode scanning (device camera; not exercised in this walkthrough — no physical/virtual barcode available in this environment).
- Clicking a product card adds 1 unit to the cart; clicking again increments the existing line by 1 (`use-pos-cart.ts`). An out-of-stock card is inert — clicking it shows an "out of stock" toast and does not touch the cart.

Cart (`POSCartPanels` / `POSCartItem`)

- **Line items** — each row shows name, unit price, a quantity stepper (Minus/Plus buttons + the qty itself, both real `<button>` s), and the line subtotal. Swipe-left-to-delete on touch devices; a trash

icon does the same on desktop. Confirmed live: added 3 distinct products across two sales, incremented one line to quantity 2 via the stepper, all totals recalculated correctly.

- **Customer selector** — defaults to "Walk-in customer"; clicking it opens a search/add-new picker (`POSCustomerSelector`). Confirmed live: created a new customer ("SmokeTest Customer") inline and it became the active customer for the sale (required for Credit payment — see below).
- **Discount** — "+ Add discount" reveals an amount input + Fixed/% selector. Confirmed live: a ₦100 fixed discount on a ₦1,450 subtotal correctly produced a ₦1,350 total, and the discount amount and type both survived a hold/recall round-trip (see Held Transactions below).
- **Hold Sale** — parks the current cart (see Held Transactions).
- **Clear cart** — confirmation dialog before wiping the current cart.
- **Charge** button opens the Payment dialog; disabled with `₦0` label when the cart is empty.
- **Redeem Reward** (`POSRedeemReward`) — this is where loyalty-point redemption actually happens (see `docs/features/customers.md` 's Loyalty tab section, which only *configures* tiers/rewards and does not itself offer a redeem action). Appears once a customer is selected and at least one active redemption option has a naira `discount_value` configured (non-monetary perks like "Free Delivery" stay configurable but aren't applicable as a POS discount). Picking an affordable option sets it as the cart's discount and shows "Redeeming: (N pts)" with a clear button; a manual discount edit or a held-sale recall both detach any active redemption (the discount and the redemption share one slot). **Gated** on `useFeatureGate().canUseLoyaltyProgram` — plan tier (Pro/Enterprise) AND the store's own "Enable Loyalty Program" toggle (Settings → Business Info, or the Loyalty Settings dialog's Program Status section) — the control is hidden entirely, not just disabled, when either is off, and any already-staged redemption is cleared automatically if the gate flips off mid-session (e.g. a plan downgrade). `usePOSPayment` also refuses to write earned/redeemed `loyalty_transactions` rows (or the sale row's `points_earned` / `points_redeemed`) when the gate is closed, as defense-in-depth against a stale UI state.

Held transactions (park & resume)

Real, working feature — not a stub. Backed by the `held_transactions` table (`lib/db/schema.ts`), `usePOSHeldTransactions` (`lib/hooks/use-pos-held-transactions.ts`), and `HeldTransactionsDialog` (`components/pos/held-transactions-dialog.tsx`).

- **Hold Sale** → `handleHoldTransaction` : serializes the current cart (`items_json`), the discount/discount_type, and the customer, inserts a row, clears the live cart, and shows "Transaction held successfully". The cart-panel header then shows an amber "N on hold" badge with a "View" link.
- **View** opens `HeldTransactionsDialog` : lists every held sale with customer name, held-at time, item count, and total; each row has **Recall** and a destructive delete (with its own pending-state spinner).
- **Recall** → `handleRecallTransaction` : clears whatever's in the cart right now, re-hydrates every line item by looking its `product_id` up in the live product list (carrying over the held quantity),

restores the discount and discount_type, restores the customer if one was set, and deletes the held row.

- Confirmed live end-to-end: held a 3-line cart (10ML SYRINGE ×2, 2ML SYRINGE ×1, ABONIKI BALM 25G ×1, ₦100 fixed discount, total ₦1,350) → cart emptied, "1 on hold" appeared → opened the dialog, saw "3 Items" / ₦1,350 → Recall → all 3 lines, quantities, and the ₦1,350 total came back exactly. Then completed the sale for cash.
- **Coverage gap found and closed this task** — see Testing section below: zero existing tests touched this feature before this task.

Payment (`POSPaymentDialog` / `payment-method-selector.tsx`)

Five payment methods, individually toggleable in Settings → Payment Methods: **Cash**, **Card**, **Transfer**, **Credit**, **Mixed**. All five were enabled for this store.

- **Cash** — Amount Paid input, pre-filled with the exact total; shows "Change: ₦X" once the amount is ≥ total. Confirmed live: paid exact amount, sale completed, receipt showed the paid/change lines.
- **Card** / **Transfer** — when "Require Payment Destination Account" is on (it was, for this store), a Destination Account dropdown appears, filtered by account type: Card only shows accounts of type `pos_terminal`; Transfer shows every other type (bank/mobile money). This store had zero `pos_terminal` accounts configured, so Card's dropdown was legitimately empty (not a bug — confirmed by adding a "Smoke Test POS Account" of type Bank and seeing it appear under **Transfer**, not Card). Added a Payment Account via Settings, then completed a real Transfer sale against "Zenith Bank POS" — receipt correctly showed `Payment type: TRANSFER`. A "Set as default for Card/Transfer on this device" checkbox appears once an account is picked.
- **Credit** — requires a selected customer first (blocked with a toast otherwise: "Please select a customer for credit sales"); on success, adds the sale total to that customer's `outstanding_balance`.
- **Mixed** — `PaymentSplits`: split the total across multiple method+amount(+account) rows; must fully cover the total before Process Payment is enabled.
- **Sale Note** — optional (or required, per a store setting) free-text note attached to the sale.
- **Process Payment** inserts the `sales` row, one `sale_items` row per cart line, deducts/logs stock per item (see Testing → bug fixed, below), applies loyalty points and redemption if a customer/reward was involved, then shows the receipt dialog and clears the cart.

Receipt (`ReceiptView` / `POSReceiptDialog`)

Shows store name, invoice number, customer, date, cashier, a line per item (qty/price/total), subtotal, discount (if any), tax (if any), grand total, and payment type/change. **Print Receipt** and **Close** buttons. Confirmed live for both a cash sale (with discount) and a transfer sale.

Product-request empty state (`RequestItemDialog`)

Mentioned in recent commit history (`cf4964fa`). `RequestItemDialog` (`components/pos/request-item-dialog.tsx`) is reachable from the cart panel's **Request Item** button at any time (not only on a true empty-cart state) — it seeds its product-name field from the current search term (`initialProductName`), lets the cashier pick/search a customer, set a quantity and notes, and calls `logRequestedProduct()` (`lib/db/requested-products-queries.ts`), writing to the `requested_products` table for the store to review later (e.g. "customer asked for a product we don't stock"). Data-layer coverage already exists in `__tests__/requested-products.test.ts` ; the dialog's UI itself has no dedicated e2e coverage (not addressed this task — see Testing below for what was prioritized instead).

Testing

Coverage gap found

```
$ grep -rln "held_transaction\|hold\b" __tests__/ e2e/ components/pos/
__tests__/date-utils.test.ts   # unrelated match on the word "hold"
components/pos/pos-system.tsx
components/pos/pos-cart.tsx
components/pos/transaction-item.tsx
```

Zero test files (Vitest or Playwright) exercised held transactions before this task, despite it being a real, wired-up feature. The product-request dialog had data-layer coverage (`requested-products.test.ts`) but no e2e coverage of the dialog itself.

Test added: `client/e2e/pos-held-transaction.spec.ts`

Targets the riskiest untested failure mode: `handleRecallTransaction` rebuilds cart lines from the held row's `items_json` by looking each item back up in the *live* products list, silently dropping any item it can't find via `.filter((item): item is CartItem => item !== null)` . A renamed/deactivated product, or one not yet loaded into that list, would shrink the cart on recall with no error — the cashier would see a smaller total, not a warning.

The test creates two fresh products, gives them stock via a real Cycle Count, builds a 2-line/3-unit cart with a fixed discount, holds it, reopens the held list, recalls it, and asserts every line item, quantity, and the discount all survived byte-for-byte (matching totals before/after), then completes the sale. Passed twice in a row against `--project=chromium --no-deps` .

While writing it, confirmed live that the Cycle Count screen's current UI (inline "Counted Qty" cells in a search+grid layout, "Review & submit" → "Submit audit") differs from the older select-a-category-then-"Start count" flow `e2e/sales-lifecycle.spec.ts` still asserts against — that flow no longer exists in the app. This is a second instance of the same pre-existing/unrelated e2e-suite drift already

logged in `docs/features/_findings-log.md` (`products.spec.ts` , `global.setup.ts`); not fixed here (out of this task's scope), but logged as a new entry.

Bug found and fixed: saving a product without a category always failed

Found while creating the two test-fixture products above (Category has no "*" in the Add Product dialog — it's documented as optional) and reproduced independently, live, via Chrome DevTools console:

```
Failed to save Product: Wrong API use : tried to bind a value of an unknown type (undef
```

Root cause: `useSaveProductMutation` (`lib/hooks/use-save-product-mutation.ts`) set `localPayload.category_id = undefined` when no category was chosen. `base-helpers.ts` 's `insert()` / `update()` bind every payload value straight into a sql.js prepared statement with no null-coalescing, and sql.js's `bind()` throws on a JS `undefined` — it only accepts `null` for an absent column. Every product ever saved without a category (the common case for a retail/pharmacy store that hasn't set up categories yet) silently failed to save, with no created row and only a generic toast.

Fix: `localPayload.category_id = null` instead. The mutation's async body was extracted to a standalone `saveProductToLocalDb()` export so it's directly unit-testable without a React/QueryClient tree.

Verified:

- `client/__tests__/save-product-no-category.test.ts` — real in-memory SQLite against the app's actual `SCHEMA_SQL` (not a mocked insert), confirmed to fail with the exact same `sql.js` error against the pre-fix code, pass after.
- Live re-test in Chrome: created "Manual UI Test Product XYZ2" with no category selected → "Product added successfully", row appeared in the catalog tagged "Uncategorized".
- `client/e2e/sales-lifecycle.spec.ts` (pre-existing, previously blocked at this exact step) now gets past product creation; it still fails further along at the stale Cycle Count assertions described above — unrelated, pre-existing, not addressed here.

Second bug found and fixed: a completed sale could leave zero trace in the stock ledger

Found while investigating unexpectedly negative stock counts during the live walkthrough (10ML SYRINGE and others went negative after ordinary test sales). Traced via direct inspection of the synced MySQL data: a sale's `sale_items` row existed with no matching `stock_movements` row at all for one of its lines.

Root cause: in `use-pos-payment.ts` 's `handlePayment` , `getBatchesForProduct()` filters to `quantity > 0` (correct for FEFO picking). If a product's real stock was already fully depleted, that filter returns an empty list — and the payment loop then had **no batch to attribute the sale to at all**: no `stock_batches` update, no `sale_item_batches` row, no `stock_movements` row. The sale still

completed and was recorded as revenue; the unit(s) sold simply vanished from the inventory ledger with no error or warning anywhere.

Fix: extracted the per-item batch-deduction logic into `recordSaleItemStock()` (`lib/db/queries/inventory.ts`), which now falls back to `getAnyActiveBatchForProduct()` (the product's most-recently-touched active batch, regardless of quantity) when no batch has positive stock, so the deduction is still attributed and logged — the batch goes further negative (same as an already-negative batch does today) instead of the sale leaving no trace at all.

Verified: `client/__tests__/record-sale-item-stock-depleted-batch.test.ts` (3 tests, real in-memory SQLite against `SCHEMA_SQL`) — confirmed to fail (`recordSaleItemStock is not a function` pre-refactor; the underlying zero-batches case reproduced separately) against the pre-fix code, pass after; also covers that FEFO picking among genuinely positive-stock batches, and the true-zero-batches edge case (nothing to attribute to at all), are unaffected.

Findings log

See `docs/features/_findings-log.md` for the running cross-task log; this task's two fixes and the stale-cycle-count-flow finding were appended there.

Prescriptions

Route: `app/(dashboard)/prescriptions/page.tsx` → `components/prescriptions/` (`PrescriptionManagement`), orchestrated by `lib/hooks/use-prescription-management.ts` (queue/detail/dispense wiring) and `components/prescriptions/new-prescription/use-new-prescription.ts` (the create/edit form). Wrapped in a `LockedModuleOverlay` at the page level.

Walked live against the "Pikarestiv Stores 2" store on 2026-09-02, including a real prescription creation and a real dispense-through-POS checkout — this store is the plan's designated real-write sandbox.

Module gate

`app/(dashboard)/prescriptions/page.tsx` renders `PrescriptionManagement` underneath a `LockedModuleOverlay` (`components/dashboard/locked-module- overlay.tsx`) with `featureKey="prescriptions"`. The overlay reads `canUsePrescriptions` from `lib/hooks/use-feature-gate.ts`:

```
canUsePrescriptions: storeProfile?.store_type === 'pharmacy'
  ? getFeature('prescriptions', 'prescriptions', true)
  : false,
```

Two independent conditions must both hold:

1. **`store_type === 'pharmacy'`** — set on Settings → Business Info → "Business Vertical" (`Pharmacy` / `Grocery` / `Supermarket` / `General`). This check is unconditional: even an Enterprise-tier store on the "General" vertical is locked out, regardless of the plan's feature flags.
2. **The `prescriptions` feature flag** for the store's subscription tier (`subscriptionPlans.tiers[tier].features.prescriptions`), defaulting to `true` for every tier if the flag is unset.

Confirmed live: Pikarestiv Stores 2 started on the "General" vertical (on a Pro trial), which locked the module with "This feature is available on the Starter plan and above" — a slightly misleading message, since the real blocker for a General-vertical store is the vertical, not the plan tier; switching tiers alone would not unlock it. Switching Business Vertical to "Pharmacy" (Settings → Business Info) immediately unlocked the module with no reload needed. Both `Overview` (Business Vertical selector) and `Billing` (the "Prescriptions, Procurement & Expenses" bullet on the Starter card) hint at this feature, but neither surfaces the vertical requirement explicitly.

Prescription queue (`PrescriptionManagement` / `PrescriptionList`)

- **Status filter chips**: All / Needs verification / Refills due / Ready for pickup / History (`PrescriptionStatusFilter`).

- **Search bar** — filters by patient name or medication; falls back to fuzzy matching (`isFuzzyFallback`) when no exact substring match is found.
- **List rows** show patient name, medication summary, and a priority badge. Selecting a row opens `PrescriptionDetailPanel` (a responsive side panel / full-screen sheet on mobile).
- **Create Prescription** button opens a full-screen overlay (`NewPrescription`) driven by the `?action=add` (or `?edit_rx=<id>` for editing) URL param.

Create/Edit Prescription form (`NewPrescription` / `use-new-prescription.ts`)

Required fields: **Patient Name**, **Phone Number**, **Doctor Name**, **License Number**, and at least one medication. Age, Priority (Normal by default), and Insurance are optional.

Per medication (Add Medication panel): **Product Name** (combobox sourced from `getAvailableStockBatches()` — i.e. only products with a stock batch `quantity > 0` are selectable), **Strength** (a second combobox populated from that same product's distinct non-empty `strength` values), **Quantity**, **Dosage** (free text, required), Unit Cost (optional — defaults to the product's catalog selling price), Refills Authorized, Refill Interval (Days, defaults to 30), and Instructions.

Confirmed live: created a prescription for "Smoke Test Patient" / Dr. Smoke Tester (MD-99999), medication TRAMADOL 100MG × 5 ("Take 1 tablet twice daily"), no refills. Saved successfully and appeared in the queue with status **Needs verification**.

No real customer link. Despite "Patient Name"/"Phone Number" looking like customer fields, the form only writes free-text `patient_name` / `patient_phone` columns onto the `prescriptions` row — there is no combobox or lookup against the actual `customers` table, and no `customer_id` foreign key. A prescription for a patient who is already a saved customer creates no relationship between the two records; the plan's Step 1 instruction to "link it to a customer" is not achievable in the current UI. Worth a follow-up if two-way prescription↔customer history is a desired feature.

Strength selector can go unusable per-product without blocking the form. `PrescriptionMedications` ' strength dropdown is populated from `availableProducts.filter(m => m.name === productName).map(m => m.strength)` — i.e. sourced from `products.strength` . For TRAMADOL 100MG (and apparently other products in this store's imported catalog), `products.strength` is blank, so once that product is chosen the Strength dropdown renders with zero options and cannot be interacted with, despite being marked required (`*`). This did **not** block adding the medication in practice: `newMedication` 's `strength` state defaults to `""` and is never touched, and the product lookup (`m.name === productName && m.strength === newMedication.strength`) still matches because the underlying batch's `strength` is also `""` . Net effect: a cosmetically-required field that silently no-ops for any product with a blank `strength` column. Logged as a UX/data-quality finding, not fixed — the underlying data-quality issue (blank `strength` on imported products) is out of this task's scope, and the matching logic already degrades safely.

Detail panel & status flow (`PrescriptionDetailPanel`)

Status progression exposed via one contextual action button, driven by `updatePrescriptionStatus` (`lib/db/queries/prescriptions.ts`):

`Needs verification` → **Process** → `In progress` → **Mark Ready** → `Ready for pickup` → **Dispense** → (hands off to POS; see below).

Confirmed live: stepped a real prescription through Process → Mark Ready → Dispense.

- **Edit** — routes to the same New Prescription overlay pre-filled via `?action=add&edit_rx=<id>` .
- **Dispense / Dispense Refill** — routes to `/pos?dispense_rx=<id>` (refill adds `&refill=1`). Does **not** deduct stock or complete the sale itself; it only marks the hand-off. See "Dispense → stock deduction" below.
- **Process Return** — looks up the prescription's linked sale (`getSaleForPrescription`) and routes to `/pos?tab=history&return_sale=<id>` so the real Return flow (which reverses stock/payment) handles it, rather than the prescription independently relabeling its own status.

Dispense → stock deduction (the Step 1 focus check)

Dispensing a prescription does **not** have its own stock-deduction logic. `handleDispense` (`lib/hooks/use-prescription-management.ts`) simply navigates to `/pos?dispense_rx=<id>` . On the POS page, `lib/hooks/use-pos-prescription.ts` (`usePOSPrescription`) watches for that query param and, once the cart is empty and products are loaded, fetches the prescription's items (`getPrescriptionItems`) and matches each one to a POS product by `name` + `strength` , loading matched items into the cart and locking it to the prescription (`Cart locked to this prescription`).

Checkout then proceeds through the **normal POS payment path** (`lib/hooks/use-pos-payment.ts`), which:

1. Calls the shared `recordSaleItemStock()` (`lib/db/queries/inventory.ts`) for every cart line — the exact same function POS walk-in sales use — to deduct stock (FEFO batch picking, with the zero-batch/partial-shortfall fallbacks already fixed in this same function during the POS and online-order smoke-test tasks).
2. On success, if `dispensedRxId` is set: calls `dispensePrescriptionRefill(dispensedRxId)` for a refill dispense, or `updatePrescriptionStatus(dispensedRxId, "completed")` for a first-time dispense.

Finding: prescription dispensing does *not* have a third independent reimplementation of the stock-deduction logic. Unlike POS checkout (originally its own copy) and online-order fulfillment (originally its own copy) — both independently found to have the same zero-batch/partial- shortfall gaps in earlier tasks and since fixed by routing through `recordSaleItemStock()` — prescription dispensing was already built by routing entirely through the POS payment path, so it inherits `recordSaleItemStock()` 's fix automatically with zero prescription-specific code to fix. **No fix needed here.**

Confirmed live end-to-end: dispensed the "Smoke Test Patient" TRAMADOL 100MG × 5 prescription — Ready for pickup → Dispense → POS cart auto-loaded and locked → Cash checkout for ₦1,500 → sale completed. The Activity Log confirmed the expected chain of writes for a prescription-linked sale (`Created a sale` → `Created a sale item` → `Updated a stock batch` → `Created a sale item batch` → `Created a stock movement` → `Updated a prescription`), and the "Updated a stock batch" entry recorded the correct post-sale quantity (147 → 142 for a batch that started at 147).

Caveat on Inventory's displayed stock for this same batch: immediately after the above, the Inventory > Catalog list and the product's Batches tab showed TRAMADOL 100MG at **-5 units** instead of 142, even after a full page reload. The Activity Log entry for the same `stock_batches` record unambiguously recorded the write as "Quantity: 142", and Stock Movements lists only the one `-5` sale movement for this product with no earlier movement explaining how it reached 147 in the first place (consistent with that starting quantity being seeded directly by a bulk import, bypassing the movements ledger, as documented in `docs/features/inventory.md` 's import finding).

A follow-up pass (fix round 1, see the task report) re-examined this and **retracted the original "cross-task interference" explanation** — the smoke-test harness that ran this task executes exactly one implementer at a time, so no second task's process could have been concurrently writing to this store; that mechanism does not exist and should never have been cited. What the follow-up actually checked and ruled out instead:

- **Not a rendering bug.** `product-batch-history.tsx` and `use-product-details.tsx` (`client/components/products/product-details/`) render `batch.quantity` straight from `getStockBatchesForProductDetails()` (`SELECT * FROM stock_batches WHERE product_id = ? AND _deleted = 0` , `client/lib/db/queries/inventory.ts`) — there is no code path that substitutes a stock movement's signed delta (e.g. the `-5` sale movement) for the batch's own stored `quantity` column.
- **Not a double-deduction.** `recordSaleItemStock()` writes an absolute `quantity: batch.quantity - deduction` via the shared `update()` helper (`client/lib/db/base-helpers.ts`), which issues a plain `SET quantity = ?` (not an increment/delta). The only delta-based writer, `updateStockBatchQuantity()` , is used solely by procurement/cycle-count adjustment code — never reached from POS or prescription checkout — so it cannot be the second write.
- **Not a cache-invalidation gap.** `queryKeys.products.batches` is tagged `meta.tables: ["stock_batches"]` , and every `update("stock_batches", ...)` call synchronously invalidates any query carrying that tag, so a stale cached pre-sale read is not the explanation either.
- **Not the sync engine.** `pushChanges()` never writes sale/stock data back into local SQLite (it only marks queue items synced or records failures), and `pullChanges()` explicitly skips overwriting a record that still has a pending entry in `_sync_queue` — which the just-made dispense update is, until it's pushed — so a pull reverting the fresh 142 is also ruled out.

Second investigation round — resolved with batch-id-level evidence. A follow-up pass re-opened this with direct access to the live sql.js database (via the dev-only `window.getDatabaseBinary()` hook in `client/lib/db/core.ts` , loaded into a fresh in-page `sql.js` instance so the binary could

be queried directly — see `docs/features/_findings-log.md`'s entry for the exact technique). This gives batch-id, product-id, and store-id ground truth the original observation never captured.

- **The leading hypothesis (multiple `stock_batches` rows on one product, one stale/negative masking the correctly-dispensed one) is ruled out.** Direct SQL against the live store confirmed that, at the time of this round, *every* product in the account — including both products named "TRAMADOL 100MG" (see below) — has exactly one non-deleted `stock_batches` row. There is no case of one product with a correct batch and a second, stale, negative batch both rendering as cards. `getStockBatchesForProductDetails`, `product-batch-history.tsx`, and `use-product-details.tsx` were re-confirmed to do exactly what the first round found (one card per row, `batch.quantity` rendered verbatim, no aggregation) — that code was never the problem.
- **The actual mechanism: two different products, in two different stores, share the exact same name "TRAMADOL 100MG".** This account's Pika Restiv user has two stores — "Pikarestiv Stores" and "Pikarestiv Stores 2" — and each has its own, completely independent product also named "TRAMADOL 100MG":
  - `products.id = baa87d56-5c5f-4e8e-8971-6fef38360f1c` (store "Pikarestiv Stores") owns `stock_batches.id = 86c3da7b-2108-4721-b497-4320a35ed728`, quantity **147**, with **zero** `stock_movements` rows ever recorded against it (consistent with the earlier note that it was seeded directly by bulk import, bypassing the movements ledger). This is the batch referenced by the original "147 → 142" Activity Log claim.
  - `products.id = 6992c53b-c17b-4870-b2f1-628f995fdc7e` (store "Pikarestiv Stores 2") owns `stock_batches.id = 73a4ded2-7776-4225-b5c6-04e98dc6a9d3` ("Opening Stock"), quantity **-5**, with exactly **one** `stock_movements` row: a `sale` of `-5` at `2026-09-02T19:47:15Z`, tied to a completed prescription ("Smoke Test Patient," `prescriptions.id = 451d3866-83e2-4a66-84f5-fdcca6136358`, `store_id` = Pikarestiv Stores 2). Because this movement is the *only* one this batch ever had, and the write is absolute (not delta), the batch's quantity immediately before the sale was **0**, not 147 — this dispense oversold a batch that had no stock, and (before bug #4's fix landed — see below) nothing floored the result, so it was correctly written and correctly displayed as `-5`.

  These are two unrelated products in two unrelated stores that merely happen to share a display name. The "147 → 142" Activity Log entry and the "-5" Inventory display documented in the first round were **never the same batch, product, or store** — there is no batch that was simultaneously 142 and -5. The most likely explanation for how the original round conflated them: this is a shared, cumulative-history test account with a store switcher defaulting to whichever store was last active, and PIN-based login in this second round itself landed on "Pikarestiv Stores" (no "2") by default rather than "Pikarestiv Stores 2" — it is easy to compare an Activity Log entry captured in one store's context against an Inventory view captured in the other's without noticing the store had changed, producing an apples-to-oranges "same product name, different numbers" illusion. This is a documentation/process pitfall of testing on a shared multi-store account with duplicate product names, not a code defect in any read, write, cache, or sync path.

- **The "-5" itself is real, and is now explained by (and was already tracked as) bug #4** (`recordSaleItemStock` 's fallback batch had no floor on oversell, `docs/features/_known-bugs.md`). The sale that produced it (`2026-09-02T19:47:15Z`) predates the commit that fixed bug #4 (`2ed46c9e` , landed `2026-09-03T13:29`), so this negative batch is a pre-fix artifact still sitting in the local database — new oversells after that fix floor at 0 instead. This -5 row was not re-verified against the post-fix code path, since it was written before the fix existed.
- **Clean reproduction confirms the display/write path is correct under normal conditions.** A fresh, single-store, single-batch product ("ZZ BUG3 TEST DRUG") was created, stocked to 20 units via Cycle Count (batch id `7f98464f-9adf-4dee-849b-2e275feaa061`), then dispensed against via a real prescription (5 units). Direct SQL confirmed one continuous batch id throughout: `0 →(cycle-count adjustment, movement` dc02d09c `)→ 20 →(prescription sale, movement` 037740c1 `)→ 15` . The Catalog list, the product's Batches tab card, and the raw `stock_batches.quantity` all agreed on **15** with no negative figure, no phantom card, and no cross-batch confusion anywhere.

Conclusion: the original "-5/142" observation is fully explained and the leading hypothesis from round one is ruled out. There is no display bug to fix in the batch-details read path. The negative number itself is a known, already-fixed issue (bug #4) whose fix simply hadn't been applied yet to that particular pre-existing row when the first round observed it.

Test coverage

- **Unit**: `client/__tests__/prescription-calculations.test.ts` covers `calculatePrescriptionItemCost` only.
- **E2E**: none existed before this task. Added `client/e2e/prescriptions.spec.ts` (login → switch Business Vertical to Pharmacy → create a product and stock it via a Cycle Count → navigate to Prescriptions → create a prescription → confirm it appears in the queue as "Needs verification"), modeled on `e2e/customers.spec.ts` . The seeded Playwright fixture store defaults to the "General" vertical with zero products, so the spec has to do both of this doc's real setup steps itself (switch vertical to unlock the module gate; grant a product real stock via Cycle Count, the only stock-granting path a fresh free-tier store can reach — Procurement is gated behind a paid tier) before it can exercise Prescriptions at all. This is baseline flow coverage only; follow-up candidates include the Process → Mark Ready → Dispense → POS handoff chain and the refill/return flows.

Customers

Route: `app/(dashboard)/customers/page.tsx` → `components/customers/` (`CustomerManagement`), orchestrated by `lib/hooks/use-customer-management.ts` (tab/URL sync, search, modal state) and `lib/hooks/use-customer-data.ts` (fetching, tier derivation, metrics, mutations).

Walked live against the "Pikarestiv Stores 2" store on 2026-09-02 — this store is the plan's designated real-write sandbox. It currently has one pre-existing test customer ("SmokeTest Customer", Bronze tier, no balance) left over from a prior task, and is on the **Pro** subscription tier.

Tabs

`CustomerTabNav` / `CustomerManagement` render four tabs, synced to the `?tab=` URL param (`handleTabChange` in `use-customer-management.ts`): Overview, Directory, Activity, Loyalty Program.

Overview (`OverviewTab`, `InsightsStrip`)

- **Insights strip**: Total Customers, Loyalty Members (customers with `points > 0`), Total Points, Avg Points/Member.
- **Customer Segmentation**: a stacked bar + legend showing what percentage of customers fall in each loyalty tier (Bronze/Silver/Gold/Platinum), derived from each customer's lifetime spend vs. the tier thresholds.
- **Retention & Engagement**: Retention Rate, Avg Visits/Mo, Average Transaction Value — all computed by `getCustomerRetentionMetrics()` (`lib/db/queries/customers.ts`) over the last 30 days, netting refunds out of revenue.
- **Add Customer** (top-right, page-level) opens `AddCustomerModal`; creating from here auto-switches to Directory and selects the new customer (`handleAddCustomer`, covered by `e2e/customers.spec.ts`).

Directory (`DirectoryTab`, virtualized with `useVirtualizer`)

- **Search** — fuzzy match (`genericFuzzySearch`) across name/email/phone. Confirmed live: a substring match ("smoke") returns the customer; a no-match query ("zzz-no-match") renders "No customers found." rather than an empty table.
- **Filter chips**: All / Has debt (`outstanding_balance > 0`) / Loyalty members (`points > 0`). Confirmed live: both narrow correctly, and both correctly return "No customers found." for a customer with neither debt nor points.
- Selecting a row opens `CustomerDetailPanel` (a responsive side panel / full-screen sheet on mobile), showing phone, address, DOB, joined date, total spent, loyalty points, last visit, and current tier badge.

- If `outstanding_balance > 0`, an "Outstanding balance" banner appears with a **Record Payment** button (opens `RecordPaymentModal`, pre-filled with the full balance). There is no way to reach Record Payment when the balance is 0 — it's only ever exposed once a customer actually owes money, which is correct (confirmed by reading `customer-detail-panel.tsx`; not independently exercisable live since the one seeded customer has no balance and this store's product catalog is currently fully out of stock from prior tasks' smoke testing, so a fresh credit sale couldn't be rung up through POS to generate one).
- **Edit Profile** opens `EditCustomerModal` (name/email/phone/address/DOB only — no way to edit outstanding balance or loyalty points directly).
- **View History** switches to the Activity tab, pre-filtered to that customer (`handleViewHistory` → `activityFilterCustomer`).

Activity (`ActivityTab`, virtualized on desktop)

- Search by customer or transaction ID; a date-range picker.
- Defaults to a rolling 30-day window ("Showing last 30 days...") unless a customer filter, a search term, or an explicit date range is active, in which case it loads full history instead of silently truncating it.
- A customer filter (arrived via "View History") renders as a removable chip ("Showing history for X" with an ✕); confirmed live that clearing it removes the scoping.
- Each row: transaction number, customer, amount, points earned, date, and the item names on that sale (truncated with "+N more").

Loyalty Program (`LoyaltyTab`, `LoyaltySettingsDialog`)

Schema-backed (`loyalty_tiers`, `loyalty_redemption_options`, `loyalty_transactions` tables in `lib/db/schema.ts`) and **is** UI-visible — confirmed live on this Pro-tier store.

- **Loyalty Tiers Configuration**: a card per tier (Bronze/Silver/Gold/ Platinum by default) showing min. spend, points multiplier, and benefits list. `getLoyaltyTiers()` falls back to a hardcoded 4-tier default (`buildFallbackTiers` in `use-customer-management.ts`) if the store has no rows yet, so this section is never empty.
- **Points Redemption Options**: cards for each active reward (e.g. "₦500 Discount" for 500 points, "Free Delivery" for 200 points). Unlike tiers, this section has **no client-side fallback** — it renders "No redemption options configured yet." until `ensureLoyaltyDefaultsSeeded()` has run at least once for the store, which currently only happens as a side effect of opening **Edit Settings** (see below). A brand-new store will see an empty redemption section on this tab until an owner/manager opens Loyalty Settings once — a minor, non-blocking UX quirk, not a data-integrity bug (logged as a finding, not fixed, since Loyalty Settings is the only current entry point to loyalty configuration and opening it is a normal part of setting the program up).
- **Edit Settings** (visible only when `canManageStockBatch`, i.e. owner/ manager roles) opens `LoyaltySettingsDialog`, with two sections:

- **Tiers**: add/edit/delete tiers (name, min spend, points multiplier, color, benefits). Deleting a tier warns that customers in it "fall back to the next matching tier."
- **Redemption Options**: add/edit/delete rewards (label, points cost, icon, optional monetary discount value). Inactive options show an "inactive" badge here but are hidden entirely from the main tab's public list (`redemptionOptions.filter(o => o.is_active)`).
- **Program Status**: a top, clearly-separated "Enable Loyalty Program" switch, backed by `stores.loyalty_program_enabled` (DEFAULT `1` /ON). Turning it off pauses the whole program — POS checkout stops earning points and the Redeem Reward control disappears — without touching any tier/reward configuration below it. The same switch (same field, same `updateStoreProfile()` mutation) also appears in Settings → Business Info, next to "Enable Online Store."

Correction (this task): an earlier finding in this file (and in `docs/features/_known-bugs.md` #2) claimed no screen in the app lets a customer redeem earned points, including POS. That was wrong — it only checked the Customers module. **POS checkout is where redemption actually happens**: the cart's Redeem Reward control (`POSRedeemReward` , see `docs/features/pos.md` 's Cart section) lets the cashier spend a customer's points against an active redemption option as a line discount, and predates this whole smoke-test session (commit `2f1abfd7`). Directory, the customer detail panel, and this tab's own screens genuinely have no redeem action — that part of the original finding was correct — but the wider "no consuming UI anywhere in the app" claim was not.

Loyalty transactions (the `loyalty_transactions` table, i.e. a ledger of points earned/redeemed) still has no dedicated UI surface in this module — points are shown only as a running balance (`customer.loyalty_points`) on the detail panel and Directory table, not as a per-event history. That part of the original observation stands; only the "no redemption anywhere" framing was corrected.

Customer payments (`customer_payments` table)

`recordCustomerPayment()` (`lib/db/queries/customers.ts`) is the single source of truth for logging a debt payment, used by both the Record Payment button on a customer (Directory) and on a specific sale's transaction details page. For each call it:

1. Inserts a `customer_payments` row (amount, method, notes, timestamp).
2. Re-reads the customer's current `outstanding_balance` from the DB (does not trust caller-held state, since it's called from more than one page) and updates it to `max(0, currentBalance - amount)` .
3. Applies the payment **FIFO** across the customer's `pending` / `partial` sales (`applyCreditPaymentFIFO`), oldest first by `created_at` : each sale's `amount_paid` is bumped and its `payment_status` flips to `completed` once fully covered, otherwise stays `partial` .

Per this task's brief, this was checked for the same class of edge-case bug found elsewhere in this codebase (a `quantity > 0` -filtered batch loop duplicated across POS checkout and online-order fulfillment): **no bug found**. Overpayment (paying more than the outstanding balance, or more than the sum of pending sales) is handled correctly — the balance clamps to 0 rather than going negative, and

the FIFO loop stops allocating once `remaining <= 0` rather than over-crediting a sale past its own total. See "Coverage gap closed" below for the test that verifies this.

The `RecordPaymentModal` UI additionally caps the amount input at `customer.outstanding_balance` via the input's `max` attribute (native HTML5 constraint validation blocks submission above it), so overpayment via this modal specifically shouldn't happen in practice — the server-side clamp in `recordCustomerPayment` is a correct defense-in-depth for the other entry point (the sale-detail page's payment button) regardless.

Coverage gap closed

```
$ grep -rln "loyalty\|customer_payment" __tests__/ e2e/
__tests__/loyalty-calculator.test.ts
__tests__/loyalty-store-scoping.test.ts
__tests__/iif-parser.test.ts
```

Loyalty had two existing test files (points-multiplier math, and store scoping of tiers/redemption options). **`customer_payments` / the payment recording flow had zero coverage anywhere** — no unit test, no e2e test — despite being money-accumulation logic touching three tables. Added `__tests__/customer-payments.test.ts` (4 tests, against a real `sql.js`-backed schema, same pattern as `customers-total-spent.test.ts` and `loyalty-store-scoping.test.ts`): partial payment deducts correctly, an overpayment clamps the balance to 0 instead of going negative, FIFO allocation across two pending sales, and FIFO not over-allocating when the payment exceeds the total owed across all pending sales. All 4 pass against the existing (correct) implementation.

Bug found and fixed: Loyalty Program tab was never gated by plan tier

`lib/hooks/use-feature-gate.ts` defines `canUseLoyaltyProgram: getFeature('loyalty_program', 'loyalty_program', isPro || isEnterprise)` — intended to restrict the Loyalty Program module to Pro/Enterprise stores, the same way `canUsePrescriptions` / `canUseProcurement` / `canUseExpenses` / `canUseAuditMode` restrict their modules via `LockedModuleOverlay`. But `canUseLoyaltyProgram` was **never referenced anywhere else in the codebase** (confirmed via `grep -rn "canUseLoyaltyProgram"`) — the Loyalty Program tab in `CustomerManagement` rendered unconditionally for every plan tier, including Free/Starter, with no lock overlay and no upgrade prompt. Full tier/redemption CRUD (`LoyaltySettingsDialog`) was reachable by any owner/manager on any plan.

Fix: extended `LockedModuleOverlay`'s `featureKey` union with `"loyalty_program"` (mapped to `!canUseLoyaltyProgram`), and wrapped the Loyalty Program `TabsContent` in `customer-management.tsx` with it, matching the existing pattern used for Prescriptions/Procurement/Expenses:

```
<TabsContent value="loyalty" className="relative flex-1 min-h-0 mt-0 border-none p-0">
  <LockedModuleOverlay featureName="Loyalty Program" featureKey="loyalty_program" />
```

```
  <LoyaltyTab tiers={loyaltyTiers} currencyCode={storeProfile?.currency} />
</TabsContent>
```

Verification: `npx tsc --noEmit -p .` is clean and the full `vitest run` suite (340 tests) still passes. Live-verified on this task's test store (Pikarestiv Stores 2, Pro tier): the Loyalty Program tab still renders fully unlocked after the fix — no regression for tiers that should have access. A live "before" repro on a Free/Starter tier would have required changing this store's subscription tier via Settings → Billing; that was intentionally not done, since changing billing/plan settings on a shared test store falls outside a routine code-verification step. The bug is confirmed instead by direct code inspection: `canUseLoyaltyProgram` had zero call sites before this fix, identical in kind to the already-established pattern (compare `canUsePrescriptions` etc., which are each read by exactly one `LockedModuleOverlay` call) that every other gated module relies on to actually enforce its lock.

Procurement

Routes (all under `app/(dashboard)/procurement/`):

- `page.tsx` → `/procurement` — Purchase Orders tab (default)
- `vendors/page.tsx` → `/procurement/vendors` — Vendors & Suppliers tab
- `requests/page.tsx` → `/procurement/requests` — Requested Products tab
- `new/page.tsx` → `/procurement/new` — Create Purchase Order (full-page form)
- `edit/page.tsx` → `/procurement/edit?id=<poId>` — Edit an existing (draft) PO

All four dashboard pages are near-identical shells: each wraps `<ProcurementManagement initialTab="orders" | "suppliers" | "requests" />` (`components/procurement/index.tsx`) in `<RequireRole>` and `<LockedModuleOverlay featureKey="procurement">`. The three tabs are one client component with tab state, not three separate pages worth of logic — `ProcurementManagement` renders `ProcurementTabNav` plus whichever of `PurchaseOrderTable` / vendor directory / `RequestedProductsTab` is active.

Access control

`components/auth/require-role.tsx` gates the route: `allowed = isAdmin || canManageStockBatch`. `checkCanManageStockBatch` (`lib/context/auth- context.tsx`) allow-lists roles `admin`, `manager`, `specialist`, `store_owner` (normalized, case/punctuation-insensitive). The Store Owner PIN account used for this walkthrough (`Pika Restiv`, role `store_owner`) is covered by both `checkIsAdmin` (matches `"storeowner"`) and `checkCanManageStockBatch` directly. Unlike the sidebar (which simply hides the link), this is server-independent, client-side enforcement in a local-first app with no backend session check — the comment on `RequireRole` notes it's "the only enforcement point" for a typed URL, stale bookmark, or quick-action href reaching `/procurement` directly. Confirmed live: an authenticated-but-disallowed session would be redirected to `/dashboard` with a toast; the Store Owner account loads the page normally.

Vendors & Suppliers tab

`components/procurement/procurement-management.tsx` (suppliers view) backed by `getSuppliers()` (`lib/db/procurement.ts`). Each row aggregates:

- `total_orders` / `total_value` — count and sum of **all** of that supplier's non-deleted purchase orders, regardless of payment status.
- `total_debt` — sum of `total_amount - amount_paid` across **unpaid** orders only.
- `last_order_date` — max `order_date` across all orders.

(This aggregation was previously hardcoded to 0 client-side; see the regression tests in `__tests__/procurement.test.ts`, `describe("getSuppliers")`.)

"Add Supplier" opens a dialog (name, contact person, email, phone, address, tax ID, payment terms, active toggle) → `createSupplier()`. The Supplier Name field is a combobox pre-seeded with a static list of common Nigerian pharma distributors (Emzor, GSK, Cipla, etc.) for quick-fill, but accepts any free-text name — confirmed live by creating "Smoke Test Pharma Ltd", a name not in that suggestion list. Selecting a supplier row opens a detail panel (Email/Phone/Total Orders/Total Value/Last Order) with **Edit Details** → `updateSupplier()` and **New Order** → `/procurement/new` with that supplier pre-selected.

Purchase Orders tab

`purchase-order-table.tsx`, backed by `getPurchaseOrders(viewerId)` (`lib/db/procurement.ts`). `viewerId` is only passed when `!checkCanViewAllActivity(user.role)` — line staff see only their own orders, managers/owners see every order in the store. Status filter pills: **All Orders / Drafts / Sent / Received / Missing Expiry** (client-side filter over the same fetched list — `poTab` is deliberately excluded from the React Query key since it doesn't change what's fetched, only what's displayed). `has_missing_expiry` is computed in SQL as: does this PO have any linked `stock_movements` → `stock_batches` row (via `reference_type = 'purchase_order'`) whose `expiry_date IS NULL`.

Creating a full order (`/procurement/new`)

Two **Order Types**, both ending in the same line-item builder UI (`po-desktop-create-view.tsx` / `po-mobile-create-view.tsx`):

- **Purchase Order** (standard) — creates the PO in `pending` /draft status with no stock effect yet; stock is only added later, when it's received. `createPurchaseOrder()` (`lib/db/procurement.ts`).
- **Immediate Purchase** — order and receipt happen atomically in one transaction (self/walk-in purchases where goods arrive on the spot, no separate "receive" step needed). `createAndReceivePurchaseOrder()` (`lib/db/procurement-receiving.ts`) — see "Receiving" below; it reuses the exact same batch/cost/expiry math as the standard receive path, just without a preceding `pending` state.

Selecting a vendor accepts "Self / Walk-in Purchase" (`supplier_id: null`, displayed downstream as `vendor_name: "Self / Walk-in Purchase"`) or any real supplier, including one just created inline via "+ Create Supplier" in the combobox. Line items are added by product search (existing catalog item) or "Create '' as new product", which opens the Quick Add Product dialog pre-filled with the typed name — this is the flow the pre-existing `e2e/procurement.spec.ts` test exercises (and stops at, without submitting).

Each line item stores `bulk_quantity` (in the product's bulk unit, e.g. "Carton"), `units_per_bulk`, and `unit_cost` (per bulk unit); `subtotal` and the order's `total_amount` are **always recomputed** server-side from `bulk_quantity * unit_cost` at save time — never trusted from a stale client-side `subtotal` field left over from an earlier quantity/cost edit in the UI

(`createPurchaseOrder()` / `updatePurchaseOrder()` , both covered by regression tests for this exact "stale subtotal" class of bug).

Walked live end-to-end on "Pikarestiv Stores 2": created supplier "Smoke Test Pharma Ltd" → Purchase Order → PARACETAMOL INJECTION × 3 Cartons @ ₦6,000 → Save as Draft → `PO-83FF168D` , ₦18,000 total. **Mark as Sent** moves `status: 'pending' → 'sent'` (`updatePurchaseOrderStatus`); no stock effect.

Receiving (`ReceivePOPanel` / "Receive Goods")

`receivePurchaseOrder(id, receivedItems?)` (`lib/db/procurement-receiving.ts`) is the function that turns a "Sent" PO into real stock. Per line item, inside one `transaction()` :

1. **Received quantity** — defaults to the PO line's ordered `bulk_quantity` , but the "Receive Goods" form lets the user override it per line (`ReceivedItem.quantity`) — this is how a **partial receive** (less arrived than ordered) is supported. The units-per-bulk conversion always uses the *product's current* `product_units_per_bulk` , not the snapshot stored on the PO line item, in case packaging was corrected since the order was placed.
2. `insert("stock_batches", ...)` — a brand-new batch row with `quantity: bulk_quantity_received * units_per_bulk` , `cost_price` (either the ordered `unit_cost` or a per-receive override), `batch_number` (a typed lot number, or the PO's own ID as a fallback), and `expiry_date` . Because a product's total stock is always `SUM(quantity) FROM stock_batches WHERE ... is_active = 1` (see `lib/db/queries/products.ts` , `getActiveProductsForPO()` , etc.) — never a single mutable counter — this insert is *inherently* additive and can't double-apply or clobber existing stock the way an `UPDATE ... SET quantity = quantity + ?` could. This is the opposite failure mode from the stock-*deduction* bug found elsewhere in this codebase (POS checkout / online-order fulfillment): here there is no shared mutable running total to get out of sync.
3. `insert("stock_movements", ...)` — `movement_type: "purchase"` , `reference_type: "purchase_order"` , `reference_id: <poId>` . Its `total_cost` is recomputed from the *actually received* quantity, not the PO line's full ordered `subtotal` — those diverge on a partial receive.
4. If a `selling_price` override was entered, `update("products", ...)` applies it immediately (lets a discovered price change be applied without a separate trip to Edit Product).
5. Once every line item is processed, `updatePurchaseOrderStatus(id, "received")` — the whole PO moves to `received` in one step; **there is no partial/"received"-with-shortfall status** in this app (only `pending` / `sent` / `received`). A short-received line is still fully valid and fully reflected in `stock_batches` / `stock_movements` — the PO's own status just doesn't distinguish "received in full" from "received, some items short."

Verified live (partial-receive path, standard PO — the one path with zero pre-existing automated coverage; see Coverage gap below): PARACETAMOL INJECTION stock was 0 before. Ordered 3 Cartons on `PO-83FF168D` , marked Sent, then received only **2** of the 3 ordered (batch `SMOKE-TEST-01` , expiry 31/12/2027). Catalog afterward showed **2 Units** in stock (not 3) and Avg Cost ₦6,000 — confirms the partial-quantity override is honored correctly and the stock addition is proportional to what was actually received, not what was ordered. PO status still shows "Received" (all 3 stages

checked) despite the shortfall, per point 5 above — **investigated as a possible bug, not one**: this matches the app's only-two-post-draft-statuses design, not a regression.

`createAndReceivePurchaseOrder()` (Immediate Purchase) is a structurally separate function that duplicates this same insert/insert/update sequence rather than calling `receivePurchaseOrder()` internally — by design, per its own doc comment, since an Immediate order has no PO row to look up yet at the point stock needs to be created (order and receipt are the same atomic step). Both paths share the packaging-conversion and cost-fallback *math* (`immediateBaseUnitCost()` / `computeImmediateLineTotal()` in the Immediate path; equivalent inline math in `receivePurchaseOrder()`), just not the insert calls themselves.

Requested Products tab

`requested-products-tab.tsx`, backed by `requested_products` (`lib/db/requested-products-queries.ts`). This is the landing page for product requests logged from **POS's "Request Item" button** (`components/pos/request-item-dialog.tsx`, labelled "Log Missing Product" — the dialog documented in Task 3's POS walkthrough) via `logRequestedProduct(product_name, customer?, quantity, note?)`.

Confirmed the link live: opened POS → **Request Item** → typed "Smoke Test Requested Product" (a name not in the catalog) → Save Request. It immediately appeared on `/procurement/requests` as **Smoke Test Requested Product**, Requested By: "Anonymous" (no customer selected), Qty: 1, 1 request, status **Pending**.

`logRequestedProduct()` de-dupes by case-insensitive `product_name` + `status = 'pending'`: a second request for the same still-pending product name increments `request_count` and adds to `quantity` on the existing row (appending to `requested_by_customer` / `notes`) instead of creating a duplicate row — this is the same "quantity accumulation on repeat requests" shape as the stock-deduction/addition bugs elsewhere in the app, but implemented as a single-row `update()`, not a batch-loop, and out of this task's stock_batches-specific scope.

The status filter (**All / Pending / Ordered**) reflects `status: 'pending' | 'ordered'`. **Mark as Ordered** (`markRequestedProductAsOrdered`) only flips this status flag — it does **not** create a purchase order, add a line item to an existing draft, or link back to any `purchase_orders` / `purchase_order_items` row. It's a manual "I've handled this, following up outside the app" acknowledgement, not an automated hand-off into the PO flow. **Delete** (`deleteRequestedProduct`) soft-deletes the row.

Test coverage

- `__tests__/procurement.test.ts` — `getSuppliers()` aggregation, `getPurchaseOrders()` (`has_missing_expiry`, viewer scoping, vendor-name fallback), `createPurchaseOrder()` / `updatePurchaseOrder()` (stale-subtotal regression), against a real in-memory sql.js schema.

- `__tests__/procurement-immediate.test.ts` — `createAndReceivePurchaseOrder()` (the Immediate Purchase path): batch/cost/selling-price behavior, cost override scaling.
- `__tests__/procurement-products.test.ts` — `getActiveProductsForPO()` cost-averaging/stock-quantity math.
- `e2e/procurement.spec.ts` — pre-existing test logs in, navigates to Procurement, opens Create Order, and exercises the Quick Add Product trigger — **stops there**, never submits/sends/receives an order.

Gap found (Step 3): `receivePurchaseOrder()` — the Standard PO path actually used by "Create Order → Purchase Order → ... → Receive Goods" (as opposed to Immediate Purchase, which is well-covered) — had **zero** automated coverage, unit or e2e, before this task. `grep -rln "purchase_order\|PurchaseOrder" __tests__/` never matched a file exercising it, and `grep -n "test(" e2e/procurement.spec.ts` shows exactly one test, which never reaches "Send" or "Receive". **PO submission and receiving had no e2e coverage.** Closed in Step 4 — see below.

Bug found and fixed (Step 5): the pre-existing `e2e/procurement.spec.ts` test was itself silently broken, independent of anything above. Commit `113a368c` ("add POItemBuilder search-to-add-row component for PO item entry") replaced the old one-item-at-a-time `POAddItemForm` — the section labelled "Add Items to Order" with a separate combobox (placeholder `"e.g. Amoxicillin 500mg"`) and explicit "Add" button — with `components/procurement/po-item-builder.tsx`: a single search box (placeholder `"Search item by name, SKU or barcode"`) that adds a row the moment a product is picked, no separate "Add" click. The old test's selectors for both of those (the "Add Items to Order" text and the `e.g. Amoxicillin 500mg` input/"Add" button) no longer matched anything, so `npx playwright test e2e/procurement.spec.ts` was failing outright — confirmed by running it as-is before making any changes here (`expect... toBeVisible failed ... locator('text="Add Items to Order"')`). Fixed by updating the test's selectors to the current UI (see the first test in the file); no application code changed for this fix, only the stale test.

Test coverage — extended (Step 4)

Added a second `e2e/procurement.spec.ts` test, "should create a standard purchase order, send it, receive it, and increase product stock", that exercises the exact path the first test stops short of:

1. Switches Order Type to "Purchase Order" (Standard) — the default is Immediate, which already has unit coverage.
2. Adds a brand-new product as a line item (the fixture DB `e2e/.auth/test-db.bin` starts with an empty catalog, so there's no pre-existing product to search for — uses the same Quick Add flow as the first test) and sets its ordered quantity to 5.
3. Saves as Draft, clicks **Mark as Sent**, then **Receive Goods** → **Confirm & Receive** (confirming the "Missing Expiry Date" warning, since no expiry was entered).
4. Navigates to `/inventory/catalog`, searches for the product, and asserts its stock now reads "5 units" — i.e. `receivePurchaseOrder()` actually inserted a `stock_batches` row and the product's derived total stock reflects it.

Both tests pass: `npx playwright test --project=chromium e2e/procurement.spec.ts --no-deps` (the brief's literal `npx playwright test e2e/procurement.spec.ts` also invokes the known, pre-existing, unrelated `e2e/global.setup.ts` breakage — not this task's to fix).

Expenses

Route: `app/(dashboard)/expenses/page.tsx` → `<ExpenseList />` (`components/expenses/expense-list.tsx`). All business logic (data fetch, search/category filtering, derived stats) lives in `lib/hooks/use-expenses-page.ts` , wrapping `useExpenseList()` (`lib/hooks/use-finance-data.ts`).

Schema

`expenses` table (`lib/db/schema.ts`): `category` , `description` , `amount` , `date` , `payment_method` , `vendor_name` , `reference_number` , `notes` , `covers_months` — plus the usual sync/soft-delete columns. `vendor_name` and `reference_number` exist in the schema but have **no UI** anywhere in the Add/ Edit dialog or detail view; they're unused columns as far as this feature goes.

There is no `is_recurring` / `recurring` boolean or cadence field. The closest concept to "recurring vs one-off" is **`covers_months`** — see below.

Plan gating

Expenses is a paid-tier feature: `app/(dashboard)/expenses/page.tsx` wraps the page in `<LockedModuleOverlay featureKey="expenses" />` , which locks the module whenever `useFeatureGate().canUseExpenses` is false — the free tier's subscription-plans config (both local and remote) explicitly sets `"expenses": false` , and the code's own fallback for an unrecognized/missing config is `!isFree` (locked for free, unlocked otherwise). Same gating shape as Procurement. See finding #9 in `_findings-log.md` for a caveat when testing this against the shared free-tier e2e fixture: the overlay takes a moment to mount after navigation, so a fast enough click can slip through before it locks the page — real for any locked module, not expenses-specific, and not something this task's scope covers fixing.

Insights strip

Four cards above the list, backed by `useExpensesPage()` :

- **Total expenses** — `sum(amount)` across every expense, **all time**, deliberately *not* smoothed (a real ledger total of cash actually spent, ever).
- **This month** — smoothed sum for the current calendar month via `getSmoothedAmountInWindow()` (`lib/db/queries/finance.ts`) — see the covers_months section below for what "smoothed" means.
- **Top category** — the category with the highest smoothed spend in the current month; shows "N/A" when there's no smoothed spend this month.
- **Transactions** — a plain count of all (non-deleted) expense rows, all time.

Category filter

`ExpenseCategoryFilter` renders **All / Rent / Utilities / Salaries / Maintenance / Marketing / Other** as chips (mobile) or tabs (desktop) — one shared `selectedCategory` state, client-side filter over the already-fetched list (`exp.category === selectedCategory` , or no filter for "All"). Verified live: selecting "Rent" against a store with only a Utilities expense correctly showed the "No expenses found" empty state; "All" restored the row.

Search (separate control, own input) filters by case-insensitive substring match against `description` only — combines with the category filter (both must match).

Add / Edit ("Add Expense" / "Edit Expense" dialog)

`components/expenses/add-expense-dialog.tsx` , one component for both create and edit (`expenseToEdit` prop switches title, button label, and whether `useSaveExpenseMutation` calls `update()` vs `insert()` (`lib/hooks/use-expense-mutations.ts`)). Fields:

- **Date** — `DatePickerInput` , defaults to today, `disableFuture` , capped to the last 5 years.
- **Category** — select: Rent / Utilities / Salaries / Maintenance / Marketing / Other. Defaults to "Rent" on Add.
- **Description** — required free text.
- **Amount** — required number, `step="0.01"` .
- **Method** — select: Cash / Bank Transfer / Card. Defaults to "Cash".
- **Spread over how many months? (optional)** — `covers_months` , a plain number input (`min="2"`). This is the "recurring vs one-off" distinction the schema actually supports: it doesn't create N separate future expense rows or repeat anything — it's a single row whose amount is *smoothed* across N months for reporting. Typing a value ≥ 2 (with an amount already entered) shows a live preview line, e.g. "Reports will count ₦1,000/month for 12 months starting September 2026, instead of the full amount in one period." Left blank/1, the full amount counts entirely in its own date's month (the ordinary one-off case).
- **Notes (optional)** — free-text `notes` column, shown only on the detail dialog when non-empty (blank notes render nothing there, not an empty section).

On edit, the form is pre-filled from the selected expense including `covers_months` and `notes` — confirmed live re-opening Edit on a just-created expense showed all fields, including the covers_months preview line, intact.

Live-verified smoothing math: created a Utilities expense, ₦12,000, `covers_months: 12` , dated today (September 2026). Total expenses (all time) showed the full **₦12,000**; This month showed **₦1,000** (12,000 / 12) and Top category flipped to "Utilities" — matches `getSmoothedAmountInWindow()` 's per-month attribution. Editing the amount to ₦15,000 (still 12 months) recalculated This month to **₦1,250** live, no page reload needed.

Expense detail dialog

`ExpenseDetailDialog` (click any row) shows amount, category badge, Description, Date, Payment Method, Recorded By (only rendered if `recorded_by_name` is present), and a Notes card (only rendered if `notes` is non-empty). **`covers_months` is not surfaced anywhere in this dialog** — a prepaid/spread expense looks identical to a one-off here; the only place its effect is visible is the Add/Edit form's preview line and the smoothed Insights-strip numbers.

Footer has **Edit** (opens the Add/Edit dialog pre-filled) and **Delete** (opens a `ConfirmDialog`: "Are you sure you want to delete this expense? This action cannot be undone." / Cancel / Delete). Confirming calls `useDeleteExpenseMutation` → `softDelete("expenses", id)` and closes the detail dialog. Live-verified: deleted a test expense, list returned to "No expenses found" and all four Insights cards reset to zero/N/A.

Desktop table: inline "quick edit"

`ExpenseDesktopRow` (desktop table only, not the mobile card list) has a pencil icon that appears on row hover. Clicking it swaps that row's Category cell for a `<select>` (options: the same six categories, no "All") and its Amount cell for an `EditableNumberCell`, plus inline Save (check)/Cancel (X) buttons — a narrower edit than the full dialog: `useQuickEditExpenseMutation` only patches `category` and `amount`, nothing else on the row. Live-verified: opening quick-edit, then Cancel, left the row's amount unchanged (no write occurred).

Virtualization

The desktop table uses `@tanstack/react-virtual`'s `useVirtualizer` over the sorted/filtered list (`DESKTOP_ROW_HEIGHT = 56`, `overscan: 8`) — noted in the task brief as already using this pattern; not something this task needed to add.

Sorting

Desktop table columns (Date/Category/Description/Method/Amount) are each sortable via `SortableHeaderCell` / `useSortableData`, toggling asc/desc per column click. Mobile card list has no sort control (always shows `filteredExpenses` in the hook's default order — dated newest first).

Test coverage

- `e2e/expenses.spec.ts` (pre-existing) — logs in, opens the Add Expense dialog, fills Description + Amount (Date pre-filled), saves, and asserts the new row appears. Never exercised Category selection, `covers_months`, `notes`, editing, or deleting an existing expense.

Gap confirmed (Step 3): `grep -n "test(" e2e/expenses.spec.ts` showed exactly one test ("should log in, render expenses page, and add an expense"). Edit and delete of an existing expense had zero e2e coverage.

Closed (Step 4): added a second test, "should edit an existing expense and then delete it", to `e2e/expenses.spec.ts`:

1. Adds a fresh expense (own fixture data, independent of the first test).
2. Opens it, clicks **Edit**, changes the description and amount, saves via **Update Expense**, and asserts the updated description is visible (and the old one is gone).
3. Re-opens the (now-updated) row, clicks **Delete**, confirms via the `ConfirmDialog`, and asserts the row disappears from the list.

Both tests pass: `npx playwright test --project=chromium e2e/expenses.spec.ts --no-deps` (the brief's literal `npx playwright test e2e/expenses.spec.ts` also invokes the known, pre-existing, unrelated `e2e/global.setup.ts` breakage — not this task's to fix).

Bugs found

None. The Step 1 live walkthrough (add with all fields, category filter, search, edit, inline quick-edit + cancel, detail view, delete) behaved correctly in every case, including the `covers_months` smoothing math on both create and edit. No fix was needed for this task.

Reports

Route: `app/(dashboard)/reports/page.tsx`. Three tabs, driven by a `?tab=` URL param (`reports` | `daily_close` | `analytics`) via `useSearchParams` / `useRouter`: **Operational Reports** and **Analytics & Insights** are admin-only (`isAdmin` from `useAuth()`); non-admins land on **Daily Close**, the only tab they can see at all. Switching tabs pushes `/reports?tab=<value>` (`{ scroll: false }`) so the tab survives a refresh. Unlike Expenses/Procurement/Loyalty, **Reports has no `LockedModuleOverlay` / `canUse*` plan-tier gate anywhere** — confirmed by reading both the page and `lib/hooks/use-feature-gate.ts` (whose `featureKey` union doesn't include `reports` / `analytics` / `daily_close` at all). Access is role-gated only (admin vs. non-admin), not tier-gated.

Operational Reports (admin only) — `components/reports/report-center.tsx`

A grid of 6 report types, each a card with **Export** (dropdown: Download PDF / Download CSV) and **Print** buttons:

- **Detailed Sales Report** (Financial) — itemized transactions with tax/ discount breakdown. Date-ranged, staff/payment-method filterable.
- **Inventory Valuation** (Operations) — current stock levels, cost value, potential selling value. *Not* date-ranged (`takesDateRange: false`) — it's a point-in-time snapshot regardless of the filter bar's date range.
- **Profit & Loss Summary** (Financial) — revenue vs. expenses. Date-ranged.
- **Customer Loyalty Report** (CRM) — top customers, points, outstanding balances. Not date-ranged.
- **Expense Categories** (Financial) — operating costs by category. Date-ranged, no staff/payment filter (`takesSalesFilters: false`).
- **Top Sellers** (Operations) — best-performing products by revenue. Date-ranged, staff/payment filterable.

All six are driven by one config table (`REPORT_CONFIG` in `lib/hooks/use-report-export.ts`) mapping each `ReportId` to its fetcher (`lib/db/queries/reports.ts`), CSV/PDF headers, and whether it takes a date range / staff+payment filter. Date columns in the exported rows (e.g. a sale's `Date`, a batch's `Nearest Expiry`) are reformatted to `dd/mm/yyyy` via `formatDateToDDMMYYYY` before export — the raw DB value (which can be a full ISO timestamp) never leaks into the CSV/PDF as-is.

Filter bar (`ReportFiltersBar`, shared with Analytics & Insights):

- **Date range** — `DateRangePicker`: dual-month calendar + 8 presets (Today, Yesterday, Last 7 days, Last 30 days, This month, Last month, Year to date, Last year) + editable `DD/MM/YYYY` text inputs. Defaults to the last 30 days on mount. Live-verified: "This month" preset correctly set the range to 1 Sep 2026 – 3 Sep 2026 (today, per the environment's clock).

- **Staff** — `StaffSelect`, sourced from `useUsers()` filtered to `is_active`, not a bespoke query — always matches the same list Settings

  > *Staff shows.*

- **Payment method** — `PaymentMethodSelect`: All methods / Cash / POS Card / Transfer / Credit / Mixed.

Export/Print, live-verified: clicked Export → Download CSV on Detailed Sales Report; a toast confirmed ("Export successful — Your CSV has been downloaded") and the file (`Sales_Report_2026-09-03.csv`) immediately appeared in the **Recent Downloads** panel with name, report type, a human-readable timestamp, and a size label (KB/MB, `localStorage`-backed via `drx_recent_downloads`, capped at the 10 most recent, scoped to this browser — not synced/shared across devices). Print opens the same generated PDF via `openBlobForPrint()` (same `generateReportPdfBlob()` the PDF download path uses) rather than a separate print-only code path, so Print and Download PDF can never drift out of sync with each other.

Daily Close — `components/reports/daily-close-report.tsx`

Visible to every role, not just admin. A single-day reconciliation view for `reportDate` (defaults to today, via a `DatePickerInput` in the page header — `disableFuture`, capped to the last 5 years — that only renders while this tab is active). Backed by `useDailyCloseData(reportDate)` (`lib/hooks/use-daily-close-data.ts`), which reads `getDailyCloseData()` (`lib/db/queries/sales.ts`) for that date's sales/returns and aggregates locally.

Four metric cards (`DailyCloseMetrics`, via `formatMetricCurrency` — see below): Total Sales, Cash Expected, Transfer/Mobile, Total Refunds, plus a fifth full-width **Total Profit (Est.)** card (`totals.total` minus summed `cost_price × quantity` across the day's line items, minus returned items' cost). Live-verified against real Store 2 data (2 Sep 2026, 3 real sales): Total Sales ₦2,950, Cash Expected ₦2,850, Transfer/Mobile ₦100, Total Refunds ₦0, Total Profit ₦1,420 — all rendered as clean whole numbers.

Payment Breakdown card — Cash / Card-POS / Transfer-Mobile / Credit Sales, each a `formatCurrency()` (not rounded) line; Card and Transfer each expand into a per-account sub-list (e.g. "Zenith Bank POS: ₦100") built from `aggregatedTotals.cardAccounts` / `transferAccounts`, keyed off the sale's parsed `payment_details.accountId` (falls back to an "Uncategorized Card/Transfer" bucket). A `payment_method: "mixed"` sale is split across its own `payment_details.splits[]` array instead of being counted once under one method — same for a `"mobile"` method value, folded into `transfer`.

Highest Selling Products card — top 5 products by quantity sold that day (not revenue-sorted), with Qty Sold and Revenue (`formatCurrency`) columns. Live-verified: TRAMADOL 100MG (qty 5, ₦1,500), 10ML SYRINGE (qty 3, ₦300), 2ML SYRINGE (qty 1, ₦50), ABONIKI BALM 25G (qty 1, ₦1,200) — matches the three real transactions on that date exactly.

Clicking Total Sales (or any payment-method card) opens a Sales List modal (`SalesListModal`) — a payment-method-filterable, searchable table of every transaction that date (time, receipt #, method, total via `formatCurrency`). Clicking a row opens the same `TransactionDetailsDialog` POS/Prescriptions use (Total Sale, Total Profit, per-line item/qty/price/ total, Print Receipt) — live-verified: reference `TXN1788373024876` , 3 line items, ₦1,350 total / ₦299 profit, both figures matched the daily aggregate.

Actions (`DailyCloseActions` , bottom of page): **Print** (browser print of the `printRef` -wrapped metrics+breakdown+top-sellers section only, not the whole page), **Export** dropdown (Download PDF via `generateReportPdfBlob` , or Download CSV via a bespoke `exportToCSV()` in the hook — a flat key/value CSV, not the same `REPORT_CONFIG` machinery Operational Reports uses), and **Cloud Sync Now**/ **Download Local Backup** in the "Daily Close Ready" banner up top (a backup of the whole local DB, not scoped to this one day — same `getDatabaseBinary()` dev/export path used elsewhere in Settings).

Analytics & Insights (admin only) — `components/analytics/`

`BusinessIntelligenceDashboard` , backed by `useBusinessIntelligenceDashboard()` → `useBIData(dateRange, { staffId, paymentMethod })` . Shares the same `ReportFiltersBar` as Operational Reports but with its own independent filter state (defaults: last 30 days, no staff/payment filter) — changing one tab's filters does not affect the other's. An **Export Reports** button runs `exportReportCsv("profit-loss", ...)` under the current filters (with its own success/failure toast), reusing the exact same CSV path Operational Reports' Profit & Loss Summary card uses.

5 key-metric cards (`BIKeyMetrics` , `formatMetricCurrency` for the 3 currency ones): Net Sales, Net Profit, Transactions (plain count), Stock Batch Value (cost-basis, not selling price), Customers (active count). Live-verified (last-30-days default range, same 3 real sales): Net Sales ₦2,950, Net Profit ₦1,420, Transactions 3, Stock Batch Value ₦1,932,908, Customers 1 — Net Sales/Net Profit exactly match the Daily Close numbers for the one day with data in range, as expected.

Five sub-tabs:

- **Sales Analytics** (default) — Revenue Trend area chart (monthly, y-axis in `₦Nk` short form via `getCurrencySymbol` + manual `/1000` , not `formatMetricCurrency`), a Product Performance table (sortable columns, `formatCurrency` per-row revenue), and a Sales by Category bar chart.
- **Profit & Loss** — a "Financial Performance Statement" walking Gross Sales → Discounts/Tax/Refunds → Net Sales → COGS → Gross Profit → Total Operational Expenses → Final Net Income (Take Home), plus Net Margin %, a Burn Distribution donut (COGS / Operating Exp. / Net Profit as % of net sales), and a Financial Health Over Time area chart. **See "Bugs found" below** — this panel's currency formatting was inconsistent with the BIKeyMetrics cards immediately above it until this task's fix.
- **Stock Batch Insights** — Critical Stock Batch Alerts (low-stock items, HIGH/MEDIUM severity chips) and a Sales by Category chart. Live-verified showing TRAMADOL 100MG at -5 units (min 10) — the same negative-stock display already flagged, not re-attributed to this task, in

`docs/features/prescriptions.md` / finding log entry "Prescriptions: dispensing does *not* have a third copy of the stock-deduction bug".

- **Customer Behaviour** — Total Customers / Loyalty Members / Avg. Transaction / Customer Retention cards, plus a Customer Purchase Patterns table (peak hours, `Math.round(avgValue).toLocaleString()` — whole-number by construction, just not routed through `formatMetricCurrency`).
- **Staff Performance** — one row per cashier (Transactions, Total Sales via `formatCurrency`, Avg Transaction via `formatCurrency` — decimals intentionally kept here, e.g. "₦983.33", since this is a per-row detail table rather than a duplicated headline figure; see the currency- formatting convention note below).

`formatMetricCurrency` vs. `formatCurrency` convention

Per the comment on `formatMetricCurrency` in `lib/utils.ts`: it exists so **headline/aggregate metric cards** read as clean whole numbers for NGN (kobo has fallen out of everyday use), while **line-item and per-row detail figures** (a single sale's total, a single product's revenue row, a per-account payment sub-total) keep `formatCurrency`'s full precision so accounting detail isn't lost. Confirmed consistent across every report this task exercised:

- Metric/stat cards → `formatMetricCurrency`: `DailyCloseMetrics` (all 5), `BIKeyMetrics` (all 3 currency cards).
- Line items/detail tables → `formatCurrency` (intentionally, per the above): `PaymentBreakdownCard`, `SalesListModal`, `TransactionDetailsDialog`, `HighestSellingProductsCard`, `ProductPerformanceTable`, `StaffPerformanceTab`, chart axis labels.

Bugs found

Profit & Loss tab: "Financial Performance Statement" showed decimal kobo precision for the same aggregate figures BIKeyMetrics rounds to whole Naira, on the same page. `components/analytics/profit-loss-tab.tsx` used `formatCurrency()` for Gross Sales, Discounts/Tax/Refunds, Net Sales, COGS, Gross Profit, Total Operational Expenses, and Final Net Income — all aggregate totals, several of them (Net Sales, Net Profit/Final Net Income) the *literal same number* already shown rounded in the `BIKeyMetrics` cards a few rows above. Live-reproduced: Net Profit read "₦1,420" in the metric card and "₦1,420.25" a few rows down in "Final Net Income (Take Home)" for the identical value, on the identical page render. This is exactly the kind of inconsistency Task 8's brief flagged (`formatMetricCurrency` "applied consistently across every report, not just dashboard/one report") — this panel was the one report where it wasn't. **Fix:** switched `profit-loss-tab.tsx`'s import and all 7 call sites from `formatCurrency` to `formatMetricCurrency`. Live-verified post-fix: COGS now shows "₦1,530" (was "₦1,529.75"), Gross Profit and Final Net Income both now show "₦1,420" — exactly matching the Net Profit metric card above them. Regression test: `client/__tests__/profit-loss-tab-currency-formatting.test.ts` (source- inspection style,

matching `dashboard-action-center-routes.test.ts`'s pattern — no component-rendering test harness exists in this repo yet). Confirmed to fail pre-fix (`formatCurrency(` present) and pass post-fix (`git stash` of the one changed file reproduces the failure).

No other bugs found. Every date-range preset, the custom `DD/MM/YYYY` text inputs, both export paths (Operational Reports' per-report `REPORT_CONFIG` CSV/PDF, and Daily Close's bespoke `exportToCSV()`), Print, and every sub-tab's data rendered correctly against live Store 2 data.

Test coverage

Before this task: zero. No `e2e/reports.spec.ts` existed, and `formatMetricCurrency` itself had a unit test (`__tests__/utils.test.ts`, pre-existing) but it didn't cover negative numbers, zero, or large values — confirmed via the brief's own gap-check command: `grep -rln "formatMetricCurrency" __tests__/ lib/ components/reports/` matched only `lib/utils.ts` (the definition) and `__tests__/utils.test.ts` (the existing, partial test) — no test under `components/reports/` at all, and no e2e coverage of any kind for this whole section.

Closed this task:

- Extended `__tests__/utils.test.ts`'s existing `formatMetricCurrency` `describe` block with negative-number, zero, and large-value cases (the brief's specifically-requested gap), plus one case documenting that a non-NGN currency's negative amount still keeps its own decimal precision (the function's `noDecimalCurrencies` set only special-cases NGN).
- Added `client/e2e/reports.spec.ts` — logs in, navigates to `/reports`, confirms the default Daily Close view renders for a non-explicit-tab visit, then switches to Operational Reports and Analytics & Insights and confirms each renders real Store 2 data (a report card grid; the BI key metric cards with a non-zero Net Sales figure) rather than an empty/error state.
- Added `client/__tests__/profit-loss-tab-currency-formatting.test.ts` (see "Bugs found" above) as the regression test for the fix.

Activity Log

Route: `app/(dashboard)/activity-log/page.tsx` → `<ActivityLogPage />` (`components/activity-log/activity-log-page.tsx`). Following the recent refactor (commit `7e28c577`), the page composes several dedicated modules under `components/activity-log/`:

- `activity-log-filters.tsx` — search box + filter pills (Date range / Action / Role / Staff).
- `activity-log-rows.tsx` — `ActivityLogDesktopTable` (div-based table, ARIA roles) and `ActivityLogMobileList` (stacked cards).
- `activity-log-detail-panel.tsx` — slide-in panel showing the full row.
- `describe-activity.ts` / `format-action-label.ts` — turn raw `action` / `table_name` values into human-readable sentences.

Data layer: `lib/db/queries/activity-log.ts` (`getActivityLog`, `getDistinctActivityActions`, `getDistinctActivityUsers`), all reading the `audit_logs` table.

Plan gating

Not gated. Unlike Prescriptions/Procurement/Expenses/Loyalty Program, `app/(dashboard)/activity-log/page.tsx` does not wrap its content in `<LockedModuleOverlay>`, and `locked-module-overlay.tsx`'s `featureKey` union has no entry for it. (The union's `"audit"` key maps to `canUseAuditMode` / `audit_mode` — a separate stock cycle-count feature, not this page — confirmed by reading both files; don't confuse the two.) Every plan tier sees the full, unfiltered Activity Log.

What generates a log entry

Every row in `audit_logs` comes from one call to `logAction()` (`lib/db/core.ts`). Two call-site shapes exist:

1. **Generic CRUD helpers** (`lib/db/base-helpers.ts`) — `insert()`, `update()`, `softDelete()`, and `remove()` each call `logAction()` automatically for **every table** they touch, so any create/update/delete that goes through the shared helpers is logged with zero extra code at the call site. Default actions: `INSERT` / `UPDATE` / `DELETE` (soft) / `HARD_DELETE` (only `remove()`, a real unrecoverable `DELETE FROM`, since this is "the one place where NOT capturing this would leave a real blind spot," per the code comment). The one exception: `feedback` inserts are deliberately **not** logged (`insert()` special-cases `table !== "feedback"`) because that table holds background crash/error telemetry, not a user action — logging it would surface every silent crash report as a "Created feedback" row.
2. **Named actions** (`lib/db/audit-actions.ts`'s `AUDIT_ACTIONS` constant) — passed as `options.action` to override the generic default when a raw INSERT/UPDATE doesn't say enough on its own:

- `LOGIN` / `LOGOUT` / `LOGIN_FAILED` / `PIN_CHANGED` — `lib/context/auth-context.tsx`.
- `RECEIVE_PO` — `lib/db/procurement-receiving.ts` (both the "receive on create" and later-partial-receive paths call it).
- `SALE_RETURN`, `STOCK_ADJUSTMENT`, `STOCK_EXPIRED`, `STOCK_DAMAGED` — defined in `audit-actions.ts`; not directly grepped for call sites here, but follow the same `options.action` pattern.

Cross-referenced via `grep -rn "logAction(" client/lib client/components`: every call funnels through the two shapes above — there is no third, ad-hoc call site anywhere in the app.

Bulk operations still log per-row

The product **import** flow inserts one product at a time through the same `insert()` helper (not a separate bulk-insert path), so a 1,513-row import produces 1,513 individual "Created a product" entries — confirmed live: with the search box scoped to `products`, entries for this task's smaller Task-1 import batch appear as one row per product, all timestamped within the same minute.

Filters

- **Search** (`SearchInput`, "Search by action, table, or staff member") — client-side fuzzy search (`genericFuzzySearch`) over `action`, `table_name`, and `user_name` **only** — it does not search the `details` JSON payload (so you can't search by, e.g., an expense's description text). When active, the page fetches up to `SEARCH_FETCH_CAP` (2,000) rows matching the other filters and fuzzy-searches that batch client-side, rather than paginating normally.
- **Date range** (`DateRangePicker`) — presets (Today/Yesterday/Last 7 days/ Last 30 days/This month/Last month/Year to date/Last year) or a manual calendar range. Maps to `from` / `to` on `getActivityLog()`, filtered server-side (well, client-side against the local SQLite DB) via `al.created_at >= ?` / `<= ?`. Verified live: "Today" correctly narrowed 2,739 rows down to the single row from today.
- **Action** — populated from `getDistinctActivityActions()` (distinct `action` values actually present for this store), labeled via `describeActionVerb()` (e.g. raw `"RECEIVE_PO"` → "Received goods for a purchase order", raw `"INSERT"` → "Created"). Verified live: selecting "Received goods for a purchase order" correctly narrowed to exactly the 3 PO-receive events on this store.
- **Role** — only shown when `canViewAll` is true (see below); options come from `STAFF_ROLES` (a fixed constant list), not from distinct roles present in the data.
- **Staff** — only shown when `canViewAll` is true; populated from `getDistinctActivityUsers()`. Verified live: selecting the single seeded user narrowed nothing further (single-user store), as expected.
- **Table** — there is no discrete "Table" filter pill in the UI (the `ActivityLogFilters` type/ `getActivityLog()` query layer supports a `tableName` param, and `getDistinctActivityActions` / `getDistinctActivityUsers` both accept an optional `tableName` too, but nothing in `activity-log-page.tsx` wires a pill to it). The closest equivalent

is typing the table name into Search (e.g. "products"), which matches on `table_name` as one of its three fuzzy fields.

All filters combine with AND semantics (`getActivityLog()` 's `conditions` array), and changing any filter resets pagination back to page 1.

Row-level visibility (`checkCanViewAllActivity`)

`lib/context/auth-context.tsx` : only `admin` / `store_owner` roles see every user's activity, the Role/Staff filter pills, and can filter by other staff. Every other role has `userId` silently forced to their own `user.id` in `baseFilters` — they only ever see their own actions, and the Role/Staff pills don't render at all for them.

Row → detail panel

Clicking any row (desktop table row or mobile card) opens `ActivityLogDetailPanel` (`ResponsiveDetailPanel` , slide-in on desktop / sheet on mobile): a human sentence (`describeActivity()`) as the title, `table_name` + formatted timestamp as the subtitle, then "Performed By", "Record ID" (if present), and "What Changed" — every key in the row's `details` JSON payload except internal/bookkeeping columns (`HIDDEN_KEYS` : `id` , `created_at` , `updated_at` , `_version` , `_synced` , `_synced_at` , `_deleted` , `store_id` , `pin`), each humanized (`humanizeKey()`) and value-formatted (`null` / `undefined` / `""` → "N/A", booleans → "Yes"/"No"). If `details` fails to parse as JSON, it falls back to showing the raw string in a `<pre>` block; if there's no `details` at all, it shows "No further details recorded." Verified live: opening a "Created a expense" entry showed Category/Amount/Description/Date/Payment Method/ Notes/Covers Months/User Id exactly matching what was entered when that expense was created.

Sorting & pagination

Desktop table columns Activity/By/When are each sortable (`SortableHeaderCell` , `ActivityLogSortKey = "created_at" | "action" | "user_name"`), toggling asc/desc per click, defaulting to `created_at desc` (newest first). Standard `TablePagination` (configurable page size, default 50) — `2,739` total rows on the live test store paginate to 55 pages at the default size.

Test coverage

Some unit coverage of the query layer existed before this task (`__tests__/activity-log-store-scoping.test.ts` , `__tests__/activity-log-filters.test.ts` — store scoping and the `tableName` filter param on `getActivityLog()`), but **zero e2e/UI-level coverage** (confirmed via `grep -rln "activity.log\|audit_logs" __tests__/ e2e/`).

Closed (Step 4): added `client/e2e/activity-log.spec.ts` — logs in, elevates to paid tier (Expenses, the traceable action used, is itself paid-tier-gated; Activity Log is not — see "Plan gating" above), adds a uniquely-named expense, navigates to Activity Log, and asserts the newest row is "Created a expense" / `expenses`, then opens its detail panel and confirms the exact Description/Amount match what was just entered and "Performed By" is a real actor (not "System").

Bugs found

None. Step 1's live walkthrough specifically set out to confirm three already-completed actions from earlier tasks left a trail — the product import (Task 1), the PO create/receive cycle (Task 6), and the expense add/edit/delete (Task 7) — and all three were found correctly logged with accurate actor, table, action, and (via the detail panel) payload. Every filter (Action, Role, Staff, Date range, Search) behaved as designed when exercised live. No audit-trail gap was found.

Settings

Route: `app/(dashboard)/settings/[tab]/page.tsx` → `SettingsPage` (`app/(dashboard)/settings/[tab]/settings-client.tsx`), driven by the `useSettings()` hook (`hooks/use-settings.ts` + `use-settings-form.ts` + `use-settings-security.ts` + `use-settings-sync.ts`). The left-rail nav (`settings-tab-nav.tsx`) groups tabs under Account / Business / Sales / Inventory / System headings; non-admin users only ever see `appearance`, `security`, and `notifications` (every other tab is gated by `ADMIN_ONLY_TABS` in `use-settings-form.ts` and silently bounces back to `appearance` if a non-admin lands on one directly).

URL aliases

Several of the 21 tab names the brief lists are **not** distinct panels — they're old/short URL aliases that `useSettings()`'s `TAB_ALIASES` map (plus two special-cased renames) redirect to a canonical internal tab, so the sidebar highlights the right item and the URL bar still shows the friendly name:

URL segment	Resolves to (internal tab)
`general`	`appearance`
`alerts`	`notifications`
`account`	`personal-info`
`store`	`business-info`
`cloud`	`data` (and opens the `CloudLinkDialog` if the store isn't cloud-linked yet)

This document gives each of the 21 brief-listed names its own `##` heading as requested, but the aliased ones point at their canonical section instead of duplicating content.

General (`general` → alias of `appearance`)

See **Appearance**, below — `general` is the exact same panel, just an older URL. Confirmed live: `/settings/general` and `/settings/appearance` render identically and both highlight "General" in the nav.

Appearance

Panel: `AppearancePanel` (`panels/appearance-panel.tsx`), composed of `theme-appearance-card.tsx`, `sidebar-preferences-card.tsx`, and `regional-settings-card.tsx`. Fields:

- **Theme Mode** — Light / Dark / System, three big buttons. Client-only (via `next-themes`, `useTheme()`), no store write.
- **Color Themes** — 6 named palettes (Dumos Blue, Ocean Breeze, Emerald Health, Ruby Retail, Midnight Gold, Professional Slate).
- **Sidebar → Expand on hover when collapsed** — a switch controlling whether the collapsed sidebar peeks open on mouse hover.
- **Regional Settings** — Currency and VAT Percentage (store-level, admin edits these under Business Info's edit form, not here — this card is read-only display of the same fields).

Verified live: toggled Theme Mode to Dark, hard-navigated to `/settings/appearance` again — Dark mode (and the selected Dumos Blue swatch highlight) survived the reload. Reverted to Light before finishing the task. This is a purely device-local (`localStorage`) preference, not a store-profile field — it does not sync across devices/staff accounts.

Account (`account` → alias of `personal-info`)

See **Personal Info**, below.

Store (`store` → alias of `business-info`)

See **Business Info**, below.

Personal Info

Panel: `AccountSettings` (`components/settings/account/account-settings.tsx`), composed of `profile-settings.tsx`, `sessions-list.tsx`, and `account-danger-zone.tsx`. Admin-only tab.

- **Personal Information** — First Name, Last Name, Email (read-only — "Emails cannot be changed"), Phone. Edit toggled via a pencil icon.
- **Sessions & Devices** — lists every login session (web / Client App / Impersonation Token) with IP and last-active timestamp, each with its own "Log out" action, plus a bulk "Log out of all other devices". This is real account session data pulled from the cloud backend, not local-only.
- **Danger Zone** — Clear Sales / Clear Logs / Clear Inventory / Clear Customers / Clear Terminals / Nuke Everything (Full Reset) / Request Account Deletion. These are destructive, irreversible, cloud-account-wide actions (per the page's own copy: "cannot be undone"). **Not exercised live** — this task's live-walkthrough store is a real, shared dev/demo account (Pikarestiv Stores 2, Pro trial), not an isolated fixture, so none of the Danger Zone buttons were clicked. This is a deliberate scoping decision, not an oversight.

Business Info

Panel: `BusinessInfoPanel` (`panels/business-info-panel.tsx`). Admin-only.

- **Business Vertical** — Pharmacy / Grocery / Supermarket / General. "Switching modes changes the terminology and active modules" per its own copy — a store-wide, disruptive change. **Not exercised live** (store was left on Pharmacy, its existing vertical) to avoid altering terminology/modules for a shared dev account mid-task.
- **Your Contact Specialist** — read-only contact card (name/email, Call / WhatsApp buttons), not a setting.
- **Business Information** (edit-toggled via pencil icon) — Business Name, Registration/CAC Number, Address, Phone, Email, Business Logo upload.
- **Store Profile** — Store URL Slug (public storefront link), **Enable Online Store** switch, **Enable Loyalty Program** switch, PCN License Number, **Include Retail Items in Suggestions** switch.

Enable Loyalty Program — next to Enable Online Store, same visual pattern. Backed by `stores.loyalty_program_enabled` (DEFAULT `1` /ON, so existing Pro/Enterprise stores saw no behavior change when this was added). Only meaningful on Pro/Enterprise plans (`canAccessLoyaltyProgramPlan` , plan-tier only — deliberately *not* the combined `canUseLoyaltyProgram` , since gating the switch's own visibility on the combined value would hide it the moment it's turned off); on a lower plan, clicking it shows an upgrade toast instead of enabling it, matching Enable Online Store's existing convention. Turning it off hides the Redeem Reward control in POS and stops loyalty point writes entirely — see `docs/features/pos.md` . The same switch (same field, same `updateStoreProfile()` mutation) also appears in the Loyalty Settings dialog (Customers → Loyalty tab → Edit Settings), for convenience.

Verified live: toggled "Include Retail Items in Suggestions" on inside edit mode, saved, hard-reloaded `/settings/business-info` — the toggle correctly showed "Enabled" after reload, confirming the write persisted to `storeProfile` . Left it enabled (a low-risk, easily-reversible setting) — noted here for anyone reconciling this store's later state.

Branches

Panel: `FleetOverview` + `MultiStoreCard` (`components/settings/store/`). Admin-only. A list-management page (Add Store / edit / delete), not a toggle-form — "Manage every store location on this account." On this store the list was empty (single-location account); not exercised further since adding a real store location is a heavier, harder-to-cleanly-revert action than a toggle.

Staff

Panel: `StaffManagement` (`components/settings/staff-management.tsx`) with two sub-tabs: **Management** (list + add/edit/delete staff, search, Role/ Status filters, CSV export) and **Activities** (`staff-activities-tab.tsx`). Admin-only. Header shows `TOTAL STAFF n / max` and `ACTIVE NOW n` stat cards, gated by `maxStaffAccounts` (`useFeatureGate`) — this store's Pro trial showed `3 / 10 max` .

Each staff row has an Edit dialog (`staff-form-dialog.tsx`): First/Last Name, Username, New Login PIN (optional, "leave blank to keep existing"), Email, System Role (`Admin (Local Master)` / `Manager (Admin)` / `Specialist (Sub-account)` / `Sales Staff / Cashier` / `Auditor (Read-only)` — from `STAFF_ROLES`), Assigned Store.

Verified live: edited "Pika Store1 Cashier2" from Sales Staff/Cashier → Specialist, saved, hard-reloaded `/settings/staff` — the Role column correctly showed "Specialist" after reload. Reverted back to Sales Staff / Cashier and confirmed the revert also persisted, leaving the store's staff roster unchanged from before this task.

e2e coverage: this is the highest-blast-radius tab in Settings (role changes gate feature access app-wide), covered by `e2e/settings.spec.ts` — see Step 4 below.

Roles & Permissions (`roles`)

Component: `RolesPermissionsPlaceholder` (`components/settings/roles-permissions-placeholder.tsx`). **This tab is not implemented** — it renders a static "coming soon" card ("Custom staff roles with fine-grained permissions are coming soon. For now, manage staff access from the Staff page.") with no fields, toggles, or forms of any kind. It is also `disabled` with a "Soon" badge in the nav rail (`settings-tab-nav.tsx` 's `NAV_GROUPS`), though the route itself still resolves fine if visited directly. There is nothing here to change or persist — confirmed live by navigating to `/settings/roles` directly.

This matters for Step 4: the brief's e2e priority list names `roles` alongside `staff` / `security` / `data` as a "change one real setting, reload, assert it persisted" target, but there is no real setting on this page to change. The e2e spec instead asserts the placeholder renders correctly and documents this explicitly rather than silently faking a persistence assertion against a static page.

Payment Methods

Panel: `PaymentMethodsPanel` (`panels/payment-methods-panel.tsx`). Admin-only.

- **Payment Methods** — five switches: Cash, Card / POS, Transfer, Credit (Debt), Mixed Payment — "Toggle which payment methods your cashiers can accept at checkout." All five were enabled on this store.
- **Require Payment Destination Account** — switch; "Force cashiers to select which bank/terminal received Transfers or Card payments."
- **Payment Accounts** — list of named destination accounts (bank/mobile money/terminal), each with its own Edit; "Add Account" button.

Verified live: toggled "Require Payment Destination Account" on, hard-reloaded — persisted as Enabled. Reverted it off afterward (its original state).

Receipt Settings

Panel: `ReceiptSettingsPanel` (`panels/receipt-settings-panel.tsx`), composed with a live receipt preview (`receipt-preview.tsx`). Admin-only.

- **Printer Paper Size** — Thermal (80mm) / A4 / Standard paper. Explicitly called out in the UI as **"This device only: a receipt printer is a property of this terminal, not your account"** — i.e. `localStorage` -scoped per browser/device, not a `storeProfile` field like the rest of this tab.
- **Store Logo** — "Managed under Business Information above" (not editable here).
- **Footer Message 1 / 2** (optional custom receipt footer text).
- **Show Logo on Receipt**, **Show Phone & Address**, **Hide "Powered by dumosrx.com"** — three switches, editable via the section's pencil icon.
- **Live Preview** — renders an actual sample receipt reflecting the current form values in real time.

Verified live: changed Printer Paper Size from Thermal (80mm) → A4 / Standard paper (no edit-mode needed — this one field is directly interactive), hard-reloaded `/settings/receipt-settings` — persisted as A4. Reverted to Thermal (80mm) afterward. Confirms the per-device/ `localStorage` scoping claim in the UI copy is accurate: this survives a reload (same device) but is a different persistence mechanism from the rest of the tab.

Register Configs

Panel: `RegisterConfigsPanel` (`panels/register-configs-panel.tsx`). Admin-only. Two switches, both directly interactive (no edit-mode gate):

- **Require Sale Notes** — "Ensure every sale includes a note before checkout can be completed." Off by default on this store.
- **Display Item Stock Levels** — "Show available stock next to each item while selling." On by default.

Verified live: toggled Require Sale Notes on, hard-reloaded — persisted as Enabled. Reverted off afterward.

Product Units

Panel: `ProductUnitsCard` (`components/settings/store/product-units-card.tsx`). Admin-only. "Add custom selling or pack units for your products, on top of the built-in list" — a tag-list CRUD (add via `+` → text input → Save; each tag has an `×` to remove). A large built-in unit list (Ampoule, Bag, Bottle, … Vial) is always available and not editable. This store already had one custom unit, "Gross".

Verified live: added a custom unit "SmokeTestUnit", hard-reloaded `/settings/product-units` — it was still present in "Your custom units" after reload, confirming the add persisted. Removed it afterward, leaving only the pre-existing "Gross" custom unit.

Categories

Panel: `CategoriesCard` (`components/settings/store/categories-card.tsx`). Admin-only. Same tag-list CRUD pattern as Product Units, for product categories used "across your catalog, inventory filters, and stock audits." This store had 26 real categories in use (Analgesics, Antibiotics, Baby Care, Beverages, … Wines & Spirits) — not exercised further live, since the underlying add/remove mechanism is identical to the one already verified on Product Units and this store's category list is real, in-use catalog data.

Alerts (`alerts` → alias of `notifications`)

See **Notifications**, below.

Notifications

Panel: `AlertsPanel` (`panels/alerts-panel.tsx`) wrapping `alert-settings.tsx` . "Stock Batch Alerts — Configure when you want to be warned about stock issues":

- **Low Stock Warning** switch — "Notify when stock hits reorder level."
- **Expiry Warning** switch — "Notify before products expire," plus a **Days before expiry to warn** number field (edit-mode gated, pencil icon
    - "Save Alert Preferences" button). This store's default was 90 days.

Verified live: changed Days before expiry from 90 → 60, saved, hard-reloaded — persisted as "60 days". Reverted to 90 afterward.

Data

Panel: `DataPanel` (`panels/data-panel.tsx`) wrapping `DataSettings` (`data-settings.tsx`) + `DataSettingsAutoSync` (`data-settings-auto-sync.tsx`) + `DemoDataSettings` (hidden unless `storeProfile.is_demo` — not shown on this real account). Admin-only.

- **Data Synchronization** status card — "Connected to Cloud" / last-synced timestamp, "Sync Now" button.
- **Background Automation** (`DataSettingsAutoSync`) — **Auto-Sync Changes** switch, and when on, a **Sync Interval** dropdown + "Save Auto-Sync Settings" button. Gated behind `canCloudSync` (`useFeatureGate` 's `cloud_sync` feature, `!isFree` fallback) — free-tier stores see this whole block dimmed/disabled with a "Pro Feature" badge.
- **Backup & Restore** — Download Local Backup, Restore from File.
- **Danger Zone** → **Factory Reset** — "Wipe all local data (products, sales, etc.) and start fresh," a "Reset All Data" button. **Not exercised live** (irreversible local-data wipe on a real dev account).

Cross-reference: Task 0's sync-queue transaction race fix

This is the UI surface for the setting whose backend race Task 0 fixed. Task 0 found `getPendingSyncItems()` (`client/lib/db/core.ts`) wasn't awaiting `awaitSettledTransactions()` (`client/lib/db/base-helpers.ts`) before reading pending sync-queue rows, so the background auto-sync timer (driven by exactly this **Sync Interval** value) could read a stale/ in-flight view of the queue. That fix is entirely in the read path this setting's timer triggers — it does not change what interval values are offered or how they're stored, which this task independently re-verified live.

On the brief's claimed 15-minute default: the literal `|| 15` fallback does exist in the current code, but it's in a different spot than the initial-state default. Two separate `15`-related fallbacks exist and it's easy to conflate them:

1. **Initial form state** (`hooks/use-settings-form.ts` line 25) — what the Sync Interval dropdown shows when the panel first mounts:

   ```
   const [autoSyncInterval, setAutoSyncInterval] = useState(
     storeProfile?.auto_sync_interval?.toString() || minimumSyncIntervalMinutes.toStri
   );
   ```

   `minimumSyncIntervalMinutes` comes from `useFeatureGate()` → `getLimit('sync_interval', isEnterprise ? 15 : isPro ? 30 : 360)` (`lib/hooks/use-feature-gate.ts` line 125) — tier-dependent: **15 min for Enterprise, 30 min for Pro, 360 min (6 hours) for Free/Starter**. No flat `15` literal here.

2. **The save handler** (`hooks/use-settings.ts` line 235, `handleSaveAutoSyncSettings`) — this is where the brief's literal `|| 15` actually lives:

   ```
   let interval = parseInt(autoSyncInterval) || 15;
   if (autoSyncEnabled && interval < minimumSyncIntervalMinutes) {
     interval = minimumSyncIntervalMinutes;
     setAutoSyncInterval(minimumSyncIntervalMinutes.toString());
   }
   updateStoreProfile({ auto_sync_enabled: autoSyncEnabled ? 1 : 0, auto_sync_interval
   ```

   In practice this `|| 15` branch is close to unreachable — `autoSyncInterval` is always a valid numeric string from the `<Select>`'s fixed option list, never blank/NaN — and even if it did fire, the very next line immediately clamps anything below `minimumSyncIntervalMinutes` back up to the tier minimum (30 for this Pro-trial store), so a stray `15` could never actually stick for a Pro/Enterprise-tier store below its own minimum... except Enterprise's minimum genuinely *is* 15, so for that one tier the fallback and the tier minimum happen to coincide.

Live-confirmed on this store (Pro trial): the Sync Interval dropdown showed **"Every 30 Minutes"** as its pre-existing value, and the dropdown's offered options were exactly {30 min, 1 hour, 6 hours} — the 5-min and 15-min options were hidden because `minimumSyncIntervalMinutes (30) <= 5 / <= 15` are both false, matching `data-settings-auto-sync.tsx`'s conditional rendering.

Verified live: changed Sync Interval from 30 Minutes → 1 Hour, clicked "Save Auto-Sync Settings", hard-reloaded `/settings/data` — persisted as "Every 1 Hour". Reverted to 30 Minutes afterward. Auto-Sync Changes was already on for this store.

Security

Panel: `SecurityPanel` (`panels/security-panel.tsx`) wrapping `security-settings.tsx`. Available to all staff (not admin-only).

- **Login PIN** — "Change PIN" opens an inline current/new/confirm-PIN form (`useSettingsSecurity` → `changePin()`). **Not exercised live** (would change the real login credential for the account being used to walk every other tab in this same session).
- **Auto-Lock Screen** — a dropdown (Off / 1 / 5 / 15 / 30 Minutes), "Lock dashboard after a period of inactivity," backed by `useAutoLockStore` (a persisted zustand store, `lib/hooks/use-auto-lock.ts`) — device-local, not a `storeProfile` field. Gated behind `canAutoLock` (`useFeatureGate`).

Verified live: changed Auto-Lock Screen from 5 Minutes → 15 Minutes, hard-reloaded `/settings/security` — persisted as "15 Minutes". Reverted to 5 Minutes afterward.

e2e coverage: covered by `e2e/settings.spec.ts` via the Auto-Lock Screen dropdown (PIN change intentionally left out of automated coverage for the same reason it wasn't exercised live above — it would invalidate the shared e2e fixture's login credential for every other spec that reuses it).

Staff (`staff`)

Already documented above, under **Staff**.

System

Panel: `SystemSettings` (`components/settings/system-settings.tsx`). Admin-only. Entirely informational / download-links — **no configurable settings live on this tab**:

- **Install App** — PWA install prompt.
- **Get DumosRx on Other Devices** — Windows / macOS / Linux / Android (APK) download links.
- **Subscription & Billing** — "Manage Billing" shortcut into the Billing tab.
- **System Updates** — Application Version (0.0.35 at time of this task), Environment (Web Browser), Platform (MacOS).
- **About DumosRx** — static licensing/support blurb.

Not exercised for persistence (there's nothing on this page a user can set).

Cloud (`cloud` → alias of `data`)

See **Data**, above. Confirmed live: `/settings/cloud` resolves to the "Data" tab (nav highlights "Data & Sync"), and — since the store used for the live walkthrough was already cloud-linked — does **not** pop

the `CloudLinkDialog`; per `hooks/use-settings.ts`, that dialog only auto-opens on the `cloud` alias when `isCloudLinked` is false.

Bug found and fixed: the dialog couldn't be dismissed on a non-linked store

Writing `e2e/settings.spec.ts` (a fresh, non-cloud-linked fixture store) surfaced a real bug the live walkthrough above never hit, since that store was already linked: on `/settings/cloud` with `isCloudLinked === false`, the "Link DumosRx Cloud" dialog reopened immediately after being dismissed (Escape or Close) — every time, with no way to actually get rid of it short of navigating away.

Root cause: `hooks/use-settings.ts`'s tab-resolution `useEffect` depended on the whole `syncState` object (returned fresh, as a new object literal, by `useSettingsSync()` on every render) instead of the one setter it actually calls, `syncState.setIsCloudLinkOpen`. Because `syncState` never has a stable identity, the effect reran on every render of `useSettings()`, and its body unconditionally re-opens the dialog whenever `internalTab === "cloud" && !isCloudLinked` — true for as long as the route stays on the alias, regardless of whether the user just closed it a moment ago. Fixed by narrowing the dependency to `syncState.setIsCloudLinkOpen` (a `useState` setter, referentially stable across renders), so the effect now only reruns when `tabParam` / `isCloudLinked` / `isAdmin` / `activeTab` genuinely change. Verified with a source-inspection regression test, `client/__tests__/settings-cloud-link-dialog-loop.test.ts` (RED pre-fix, GREEN post-fix, following the same source-inspection pattern `profit-loss-tab-currency-formatting.test.ts` established, since no component-rendering harness exists in this repo yet), and re-verified in `e2e/settings.spec.ts`'s data/cloud test, which closes the dialog and asserts it stays closed on a subsequent render before moving on.

Billing

Panel: `BillingSettings` (`components/settings/billing/billing-settings.tsx`), three sub-tabs: **My Subscription** (`subscription-plans.tsx` / `subscription-plan-card.tsx` / `subscription-status-alert.tsx`), **Billing History** (`billing-history.tsx`), **Referral Program** (`referral-tab.tsx`). Admin-only.

- **My Subscription** — Monthly/Yearly toggle, coupon code field, and four plan cards (Free / Starter / Pro / Enterprise) with per-plan feature bullets and pricing; the current plan is marked "CURRENT PLAN" with a disabled "Current Plan" button, others show "Subscribe"/"Downgrade". This store showed a **"Pro Trial (8 Days Remaining)"** status banner above the plan grid.
- **Billing History** / **Referral Program** — not walked in depth (no destructive/state-changing controls found on a skim; out of scope for this task's time budget given 21 tabs to cover).

Not exercised live: subscribing, downgrading, or applying a coupon are real billing/purchase actions (even on a trial account, these can trigger real payment-provider flows) — out of scope per this task's operating rules around financial actions. Documented read-only.

Step 3: Test coverage gap (pre-existing)

```
$ grep -rl "settings" client/e2e/ client/__tests__/
client/e2e/prescriptions.spec.ts
```

That one hit is `prescriptions.spec.ts`'s `page.goto('/settings/business-info')` call used incidentally to seed a PCN license number for an unrelated prescriptions test — it exercises no Settings behavior itself. **Before this task, there was zero dedicated e2e or unit coverage for any of the 21 Settings tabs** — the single biggest, and only fully-empty, coverage gap found across this whole smoke-test project. See `client/e2e/settings.spec.ts` (added by this task) for what's now covered.

Step 4: e2e coverage added

`client/e2e/settings.spec.ts` covers, by blast radius (not tab count, per the brief):

1. **Staff** — edit a staff member's System Role, reload, assert it persisted.
2. **Security** — change the Auto-Lock Screen interval, reload, assert it persisted.
3. **Data / Sync** — elevate to Pro tier (via the same `window.__e2eSetSubscriptionTier` dev-only hook `expenses.spec.ts` established, since Auto-Sync is a paid-tier feature and the shared fixture DB is deliberately free-tier), change the Sync Interval, save, reload, assert it persisted. This directly exercises the UI surface for Task 0's sync-queue transaction race fix.
4. **Roles & Permissions** — asserts the placeholder ("coming soon" copy) renders, since there is no real setting on this page to change (see the **Roles & Permissions** section above).

Deliberately scoped out, and called out here rather than silently skipped: purely cosmetic tabs like `appearance` / `general` (theme/color/ sidebar preferences — device-local, no state that matters to the business), and the read-only/informational `system` tab. This matches the brief's explicit instruction to prioritize by blast radius, not tab count.

Authentication / Login

Routes: `app/login/page.tsx` → `app/login/use-login-page.tsx` (tab/mode orchestration) → `components/auth/lock-screen.tsx` (account-tile PIN pad) / `components/auth/traditional-login-form.tsx` (username+PIN "Sign In" form). Session state lives in `lib/context/auth-context.tsx` (`AuthProvider` , `useAuth()`); active-store state lives in `lib/context/store-context.tsx` (`StoreProvider` , `useStore()`); auto-lock state is a separate Zustand store in `lib/hooks/use-auto-lock.ts` . Local-first: every check here is a plain SQLite (sql.js) read/write on-device (`getUserByUsernameOrEmail` , `getUserPin` , `updateUserPin` in `lib/db/queries/auth.ts`), not a network call to the Laravel backend — the backend is only involved for the subscription-plan and PIN-recovery-sync checks described below.

Walked live against "Pikarestiv Stores 2" / "Pikarestiv Stores" (both on the Pika Restiv / Store Owner account, PIN `1111`) on 2026-09-03.

The clock-in / PIN-entry screen

`LockScreen` → `UserSelection` (`components/auth/user-selection.tsx`) is a Netflix/Apple-TV-style avatar picker: one tile per entry in `localStorage.dumos_recent_users` (max 5, deduped by username, newest first — see `login()` 's update logic below), each showing first name + role, plus a dashed "Someone else" tile that hands off to `TraditionalLoginForm` 's full username/PIN "Sign In" form. With exactly one recent user the copy reads "Select your profile to continue"; with more than one it reads "Who's clocking in?" — this is the literal clock-in screen named in this task's brief. **Confirmed live: this device currently has 3 recent accounts** (Pika Restiv / Store Owner, plus two pre-existing "Sales Staff" fixture accounts — "Pika Store 1 Cashier 1" / "Pika Store1 Cashier2" — left over from the Settings task), so the 3-tile + "Someone else" grid rendered exactly as coded. Selecting a tile shows `PinEntry` (`components/auth/pin-entry.tsx`): a 4-digit `InputOTP` that auto-submits on the 4th digit (no explicit "Unlock" click needed) via `attemptLogin` , which takes the typed value directly rather than reading `pin` from closure state (the codebase's own comment: `PinPad` 's `onSubmit` can fire before React applies the last keystroke's state update).

There is no separate "store selection" step on this screen — which store is active is orthogonal to which user is logging in (see Store/account switching below); a staff member with a fixed `store_id` always resolves to that store regardless of what was last active on the device.

Correct PIN

Confirmed live twice (once via the lock-screen tile flow, once via `TraditionalLoginForm` 's username+PIN form): a correct PIN calls `login(username, pin)` (`auth-context.tsx`), which matches `dbUser.pin === pin` on the local `users` row, sets `user` state, persists `dumos_user` / `dumos_recent_users` to `localStorage` , marks `sessionStorage.dumos_session_authenticated = "1"` , force-unlocks the auto-lock store, logs

`AUDIT_ACTIONS.LOGIN`, shows a "Welcome back, `name`!" toast, and lands on `/dashboard`. Live: `DashboardOverview`'s own loading skeleton (documented in `dashboard.md`) was visible for a moment immediately after login — genuinely reproduced this time, unlike the Dashboard task's "not independently reproduced" note, since login always starts from an empty React Query cache (see Item #7 investigation below for why that matters).

Incorrect PIN

Confirmed live from both entry points. On a PIN mismatch, `login()` logs `AUDIT_ACTIONS.LOGIN_FAILED` (`reason: "invalid_pin"`, attributed to the target account's `record_id` since there's no session yet to attribute to), clears the PIN field, triggers the OTP boxes' shake animation (`hasError` → a 0.4s `x` keyframe transform), and shows an "Invalid PIN. Please try again." (lock screen) / "Invalid credentials. Please try again." (traditional form) toast. The user stays on the same screen — no redirect, no account lockout after repeated failures observed (no attempt-count throttling in the code beyond the PIN-recovery-sync cooldown described next).

Notable non-obvious behavior (read in code, not something a wrong-PIN click can trigger directly, and not exercised live since it needs a real cloud-linked store and a genuine web-dashboard PIN reset): on a mismatch, if this device has a cloud link (`auth_token` in `localStorage`) and hasn't done so in the last 10s, `login()` pulls a sync once and rechecks the PIN before actually failing — a PIN reset made via the web dashboard writes straight to the cloud DB, and this closes the "device hasn't synced down yet" gap without requiring the locked-out user to know to reload first.

Logout

Two distinct actions, both reached via the account menu in the sidebar footer (avatar + name + role, bottom-left) — confirmed live:

- **Switch Account** (`lib/hooks/use-account-actions.ts`, `handleSwitchAccount`) — calls `useAutoLockStore.getState().lockForSwitch()` and nothing else. Explicitly documented in the code as "not destructive: the local session/data stays intact, this just shows the same lock screen used for idle re-auth, forced to account-selection mode." No confirmation dialog (nothing is being cleared). This reuses the same `LockScreen` UI as auto-lock, just pre-populated with the account grid instead of defaulting to the current user's PIN entry.
- **Log out completely** (`handleFullLogout` → `performFullLogout`) — a real, destructive session end: clears `dumos_recent_users` from `localStorage` ("Clear lock screen history" — deliberately more than plain `auth-context`'s `logout()` does, per the function's own comment, which flags that a past drift once let one surface's logout leave the tile cache intact so it looked identical to Switch Account instead of actually ending the session), then calls `logout()` (clears `user`, `dumos_user`, `dumos_session_authenticated`, `queryClient.clear()`), then `router.push("/login")`. **Confirmed live:** if there are pending unsynced offline transactions (`getSyncQueueCount()`), a confirmation dialog appears first — "Unsynced Changes Detected... You have N offline transactions pending sync. If you log out now, another user logging into this

device will sync them on their account. Are you sure you want to sign out?" (`components/dashboard/unsynced-logout-dialog.tsx`) — a real, well-designed guard against silently attributing one user's offline sales to whoever logs in next on a shared terminal. Confirming lands on `/login`'s full "Sign In" form (not the tile picker, since `dumos_recent_users` was just wiped) — correctly distinct from Switch Account's tile-picker landing.

Caveat observed live: immediately after confirming "Sign Out Anyway," the still-mounted `/dashboard` route briefly rendered a broken/empty shell — empty stat-card skeleta with no shimmer, "Good morning," with no name, a "Log in again" link where the account chip normally is — for well under a second before the client-side redirect to `/login` completed. Not a data leak (no real figures were shown, just empty containers) and not the item #7 symptom (this is the opposite direction: a blank future state, not a stale past one), but a rough edge worth a follow-up: the redirect effect apparently fires after an intermediate render with `user: null` rather than before one, producing one visible broken frame.

Protected routes when logged out

Confirmed live: after "Log out completely," navigating directly to `/settings/staff` (typed URL, not a link click) did **not** show Staff Management, and did **not** immediately redirect to `/login` either — instead it briefly rendered the Settings shell itself with only the non-account-gated tabs visible (General/Security/Alerts; Business Info, Staff, Payment Methods, etc. were absent from the sidebar, and there was no account chip top-right), landed on the General tab's cosmetic Appearance/Sidebar controls, and only then completed a client-side redirect to `/login`. No real business data (staff names, store figures) was ever exposed during that window — worth documenting as a UX rough edge (a stripped shell flashes before the redirect, rather than the redirect happening synchronously/server-side), not a data-security bug, since `output: export` (this app has no server-rendered auth gate — see `inventory.md` / `dashboard.md`'s repeated mentions of `generateStaticParams`) makes a hard server-side redirect impossible here by construction.

Store / account switching (multi-store, multi-staff)

This account has both switching mechanisms reachable, and both were exercised live:

- **Store switcher** (`components/dashboard/header-store-switcher.tsx`, top-left of the header) — renders as a plain label when `availableStores.length <= 1`, or a dropdown (checkmark on the active store) once there's more than one. **Confirmed live: this account has two stores** — "Pikarestiv Stores 2" and "Pikarestiv Stores" — so the dropdown rendered. Selecting a different store calls `switchStore()` (`store-context.tsx`), covered in detail in the Item #7 section below.
- **Switch Account** (multi-staff-PIN switch) — see Logout above; this device has 3 real accounts (owner + 2 sales-staff fixtures). Selecting a different account and entering its PIN goes through the normal `login()` path (PIN-checked, cache-cleared, audit-logged), not a lighter "switch" code path — from the app's perspective this is just another login, whose target user happens to have a fixed `store_id` that may differ from whatever store was previously active. **Confirmed live:**

logging in as "Pika Store 1 Cashier 1" (a Sales-Staff account fixed to "Pikarestiv Stores") correctly hid the Store Owner's Action Center and admin-only nav items (Procurement/Expenses/Reports/Activity Log/Settings) and swapped Quick Actions to the cashier's smaller set (New Sale, Close Register, Scan Barcode, Customers) — role scoping is correctly re-evaluated on every login, not just on first mount.

Session / auto-lock interaction

`lib/hooks/use-auto-lock.ts` — a Zustand store persisted to `localStorage.dumos_autolock` (`duration`, `isLocked`, `lastActivity`). Settings > Security's "Auto-Lock Screen" dropdown (documented in `settings.md`) sets `duration` in minutes; an inactivity-polling `setInterval` (every 10s, checking `Date.now() - lastActivity` against `duration`, gated on `canAutoLock` — a paid-tier feature) calls `lock()` once exceeded. **Confirmed live via the manual trigger** (Ctrl/Cmd+L, `useLockShortcut` — the same `lock()` call the idle timer itself uses, so this is a faithful substitute for waiting out the real 5-minute default): triggering it instantly overlays the same `LockScreen` / `PinEntry` UI used for a fresh device landing, defaulting straight to the current user's PIN entry (not account selection, since `forceAccountSelection` is only set by `lockForSwitch()`, never by `lock()`). **This is a quick-unlock, not a full logout**: `user` state, `dumos_user`, and `sessionStorage.dumos_session_authenticated` are all left untouched by `lock()` — only `isLocked: true` is set — so re-entering the correct PIN just calls `unlock()` via the normal `login()` success path and returns straight to the exact same dashboard state, no re-fetch-from-scratch weirdness beyond what any `login()` call already does (see Item #7). A wrong PIN at this screen behaves identically to a wrong PIN at initial login (shake + toast, stays locked).

Item #7 investigation: does an account/store switch show stale data?

Background: `docs/features/_known-bugs.md` item #7 is a user report — "a small lag when one switches account or something where for a second or so, after switching, the old account's dashboard is shown" — with a candidate root cause already written up: `store-context.tsx`'s `switchStore()` calls `queryClient.cancelQueries()` then `queryClient.invalidateQueries()` (broad, deliberately untargeted — the code's own comment explains switching stores is infrequent enough that correctness matters more than avoiding a refetch), and `invalidateQueries()` alone marks queries stale and triggers a background refetch **without** clearing what's currently rendered — by default a component keeps showing its last-known (now-stale) data until the refetch resolves. That is a real, documented React Query behavior, and would produce exactly the reported symptom if the refetch takes long enough to notice.

What was actually tested live, three separate ways:

1. **Store switcher** (`switchStore()`'s `invalidateQueries()` path, directly matching the candidate root cause) — switched between "Pikarestiv Stores 2" (1,514 products, ₦1,918,058 inventory value, 0 sales) and "Pikarestiv Stores" (1,526 products, ₦7,501,375, 2 real sales in Recent Activity) **6 times**, alternating directions, using `browser_batch` to fire the click and take 3–4

screenshots back-to-back with no manual delay in between (the fastest round-trip this tooling allows, well under a second total). **Every single screenshot, including the very first one taken immediately after the click, already showed the fully correct new store's numbers** — Total Products, Inventory Value, Action Center card counts (which differ meaningfully between the two stores — e.g. "2 Items Expiring" only exists on one of them), and Recent Activity rows all matched the destination store on first paint. The old store's data was never observed rendering under the new store's header, at any point across 6 trials.

2. **Multi-staff account switch** (`login()` 's `queryClient.clear()` path — a *different* code path from `switchStore()` , and structurally can't show stale data by design: `clear()` removes cached data outright rather than marking it stale-but-displayable, so a slow refetch would show a loading skeleton, not old data) — switched from Store Owner (Pikarestiv Stores, admin dashboard) to "Pika Store 1 Cashier 1" (fixed to the same store, Sales-Staff role). Same rapid-screenshot technique: first post-login screenshot already showed the correct role-scoped UI (Action Center gone, nav items reduced, Quick Actions swapped) with the same store's correct figures — no stale owner-view flash, and also correctly *not* a generic loading skeleton once first rendered (only briefly visible right after the very first raw login, per the "Correct PIN" section above).
3. **Console/network check during switches:** `read_console_messages` showed no errors at any point across all switches — only expected `[StoreContext]` sync-related logs. `read_network_requests` found no relevant entries to inspect, because dashboard/store-profile data in this app is read from local sql.js, not fetched over HTTP — the store-switch refetch this task needed to time is a synchronous-feeling in-process query, not a network round-trip with its own latency budget.

Conclusion: (c)/(a) — could not reproduce the reported flash as a live, observable defect; the candidate mechanism is real but appears to resolve too fast to notice in this environment. `invalidateQueries()` 's stale-then-refetch window is confirmed to exist by reading the code (it's not a misdiagnosis), but empirically, every refetch in this app's local-first architecture (sql.js reads, no network round-trip) completed fast enough that no intermediate frame with the old store's data was ever caught — not in six single-store-pair trials, and not via the structurally different multi-staff path either. This reads as confirmed React-Query stale-while-revalidate behavior that is, in practice on this store's data volume and this test device, imperceptible — "working as designed," not a data-correctness bug — rather than a confirmed reproduction of the user's report.

What this does *not* rule out, stated plainly rather than glossed over: the user's report may reflect conditions this session couldn't reproduce — a slower device, a much larger local database (sql.js query cost scales with row count; this store's ~1,500-product catalog may simply query fast enough that the window is sub-frame), a real network-sync race (switching stores also kicks off a background `sync()` call, which several console logs showed firing concurrently with the switch — a slow/contended sync on a real device could plausibly stretch the same refetch that resolved near-instantly here), or simply a one-off perceptual effect (the toast/motion animation drawing attention right as the switch happens). None of these could be safely fabricated or tested further within this task's read-only, no-app-code-changes scope.

Suggested follow-up if this is ever reported again with reproduction steps: capture a screen recording (not manual screenshots — the ~1s window this report describes could still be shorter than this session's screenshot round-trip latency, meaning a real flash could have been missed between calls) on the reporter's actual device, and/or add a `placeholderData`/skeleton state to the store-switch transition specifically (distinct from the initial-load skeleton already on `DashboardOverview`) so that *if* a slow environment ever does hit the stale window, the user sees a neutral loading state instead of either the old store's numbers or a jarring pop-in — a UX improvement worth scoping regardless of whether the underlying report reproduces, since `invalidateQueries()`'s stale-render window is real by design even if this session's evidence says it's currently fast enough not to matter.